Chapter 6
PHP (Hypertext Preprocessor)

College of Information and Communication Technology
Can Tho University
Dr. Trieu Thanh Ngoan

Content

1. Introduction
2. PHP language basics
3. PHP functions
4. OOP in PHP
5. PHP and forms
6. PHP and MySQL
7. Cookies and sessions
8. Advanced PHP techniques (file upload, pagination, AJAX, etc.)
9. Appendix

Content

1. Introduction
2. PHP language basics
3. PHP functions
4. OOP in PHP
5. PHP and forms
6. PHP and MySQL
7. Cookies and sessions
8. Advanced PHP techniques (file upload, pagination, AJAX, etc.)
9. Appendix

What is PHP?

PHP is a server-side scripting language:

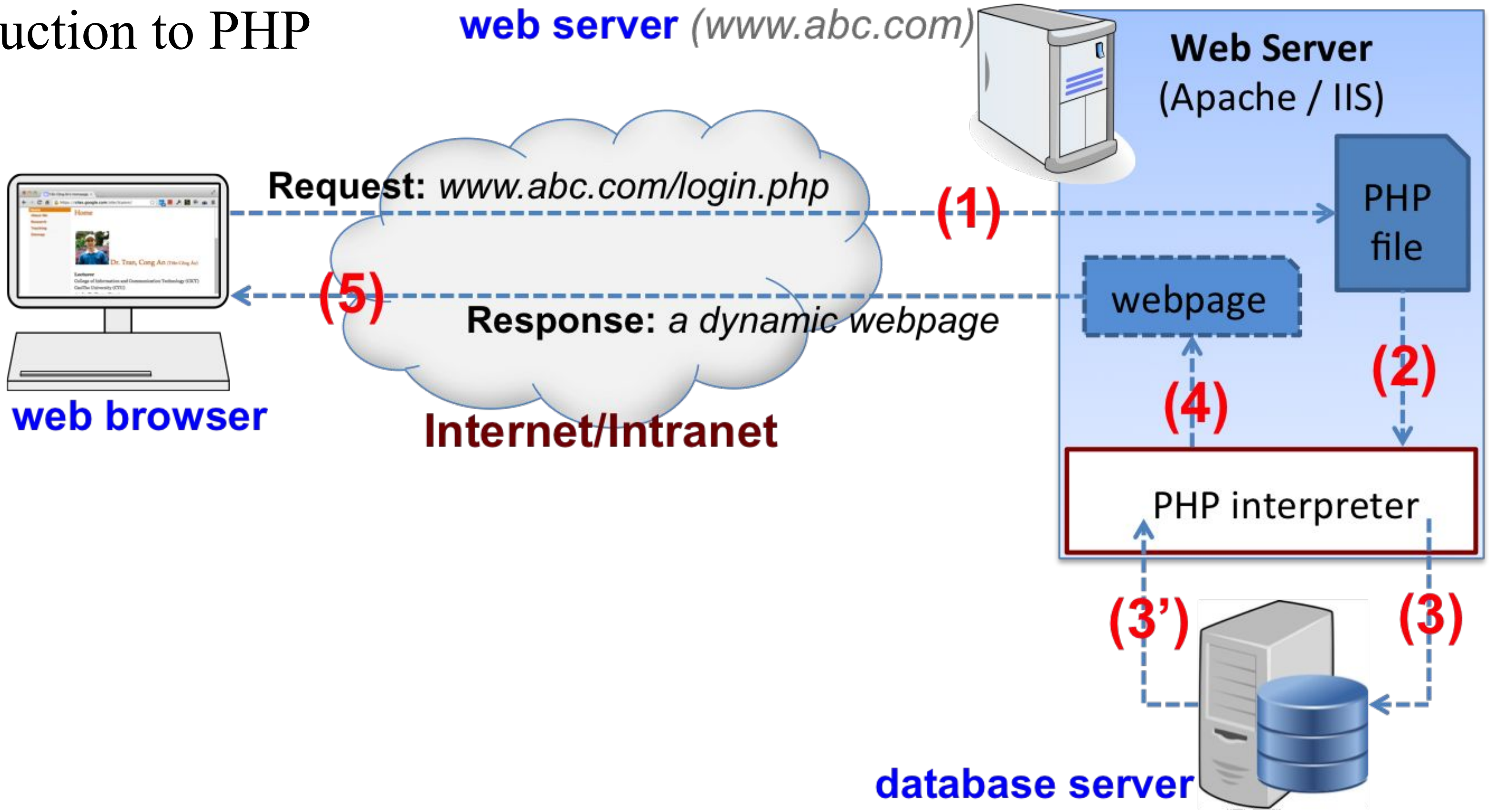
- Scripts are embedded in HTML documents
  (or: scripts to generate HTML)
- They are processed by server before returned to browser

Basic characteristics:

- Widely used and open source scripting language
- Executed on the server
- Dynamically typed and purely interpreted
- Supported by most of popular web servers (IIS, Apache, etc.) and OS (Windows, Linux, MacOS, etc.)

What is PHP?

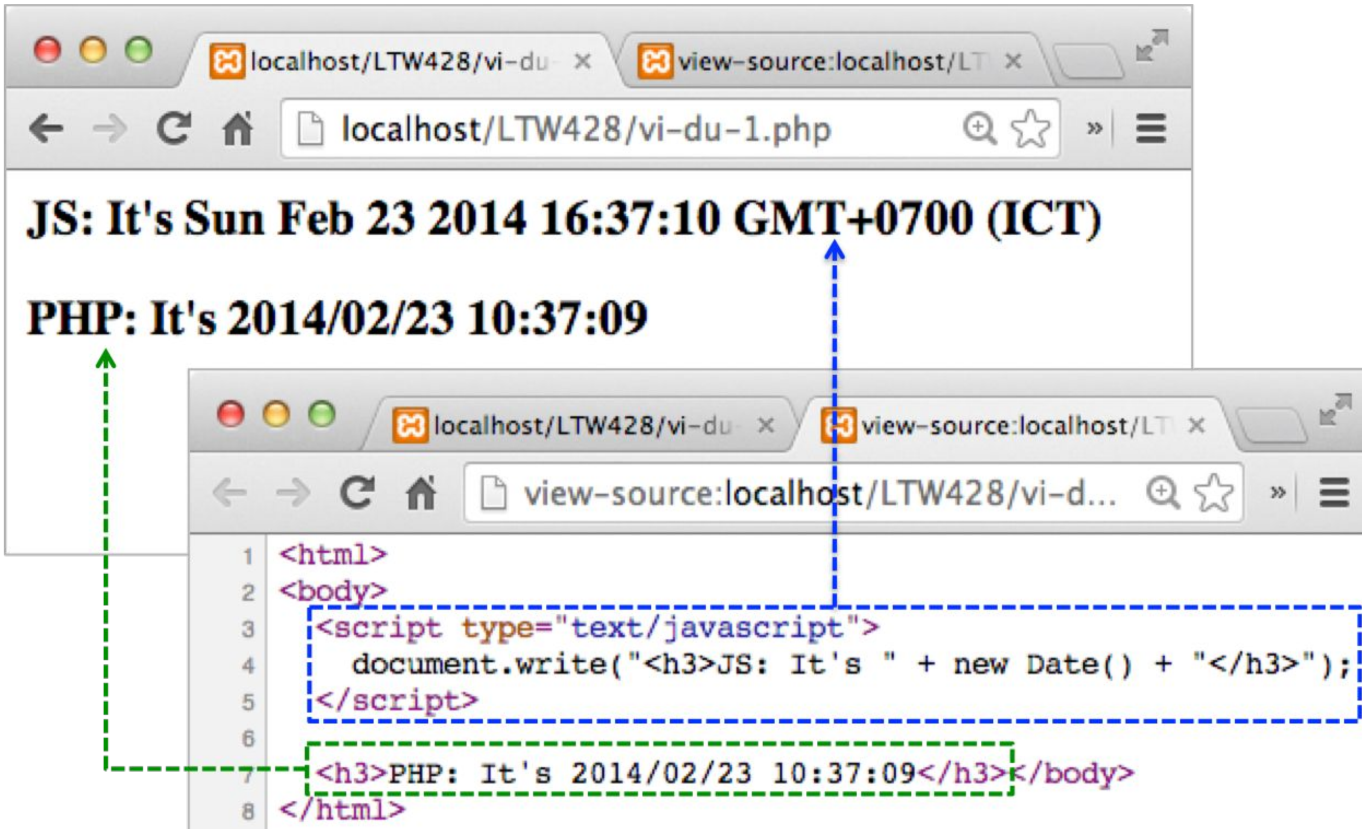
Introduction to PHP



PHP vs. JavaScript

```
//example1.php
<html>
  <body>
    <script>
      document.write("<h3>JS: It's " + new Date() + "</h3>");
    </script>
    <?php
      echo("<h3>PHP: It's " . date('Y/m/d H:i:s') . "</h3>");
    ?>
  </body>
</html>
```

PHP vs. JavaScript



What Can PHP Do?

PHP can generate dynamic page content

PHP can create, open, read, write, delete, and close files on the server

PHP can collect form data

PHP can send and receive cookies

PHP can add, delete, modify data in your database

PHP can be used to control user-access

PHP can encrypt data

Etc.

Why PHP?

PHP runs on various platforms (Windows, Linux, Unix, MacOS, etc.)

PHP is compatible with almost all servers used today (Apache, IIS, etc.)

PHP supports a wide range of databases

PHP is free

PHP is easy to learn and runs efficiently on the server side

PHP Development Environment

Suggested development tools:

- Web server: Apache (http://httpd.apache.org/download.cgi)
- PHP interpreter (http://www.php.net/downloads.php)
- DBMS: MySQL/MariaDB (http://www.mysql.com/downloads/)

Setting up the development environment:

- Option 1: download and install the above tools separately and configure them to let them to be able to "talk" to each other
- Option 2 (recommended): install a software that packages all the above software (e.g. XAMPP, AMPPS, etc.)

Content

1. Introduction
2. PHP language basics
3. PHP functions
4. OOP in PHP
5. PHP and forms
6. PHP and MySQL
7. Cookies and sessions
8. Advanced PHP techniques (file upload, pagination, AJAX, etc.)
9. Appendix

Basic Syntax

A PHP file normally contains HTML tags, and some PHP scripting code (default PHP file extension: .php)

PHP script can be placed anywhere in the document

A PHP script starts with <?php and ends with ?>

```
<?php
  //php code goes here
?>
```

PHP statements end with a semicolon ;

Variable names are case sensitive, others are case insensitive

The PHP processor has two modes: **copy** (HTML) and **interpret** (PHP)

Basic Syntax

Copy & Interpret

```
<html>
  <body>
    <h1>My first PHP page</h1>
```

← copy (HTML)

```
    <?php
      echo "Hello World!";
    ?>
```

← interpret (PHP)

```
  </body>
</html>
```

← copy (HTML)

PHP interpreter

```
<html>
  <body>
    <h1>My first PHP page</h1>
    Hello World!
  </body>
</html>
```

Web server response

My first PHP page

Hello World!

Variables

Variable can be used without declaration (dynamically typed)

Naming rules:

- A variable start with a $, followed by the variable name
- A variable name starts with a letter or the underscore
- A variable name can contain only alpha-numeric characters and underscore

```php
<?php
  $txt = "Hello world!";
  $x = 5;
  $y = 10.5;
?>
```

Variables

Scope (the parts of script where the variable can be used):

Local:

Created within a function

Can be accessed within that functions

Global:

Created outside any functions

Can only be accessed outside a function

The `global` keyword is used to access global variable within a function

All global variables can also be accessed by the associative array `$GLOBALS`

Variables

```php
<?php
    $x = 5;
    $y = 10;

    function myTest() {
      global $x;
      $GLOBALS['y'] = $x + $GLOBALS['y'];
    }

    myTest();
    echo $y;
?>
```

Result: 15

Variables

Static variables:

- Created inside a function
- Not deallocated/deleted when the function is completed (i.e. their values are retained for further usage in the latter runs)

Variable datatypes:

- Automatically assigned depending on its value (dynamically typed)
- To get the datatype of a variable: `gettype(var_name)`
- Functions to check variable type: `is_bool()`, `is_int()`,`is_float()`, `is_double()`, `is_string()`, `is_object()`,`is_array()`, `is_numeric()`, `is_resource()`, `is_null()`,`isset()`, `empty()`

Variables

```
<!-- datatype.php -->
<?php
    $f = 123.4;
    $s = "Hello world";
    $i = 100;
    echo("<p>");
    echo(gettype($f) . "<br>");  //double
    echo(var_dump($s) . "<br>"); //string(11) "Hello World"
    echo(is_int($i) . "<br>");   //int
    $i = 123.45;
    echo(var_dump($i)); //float(123.45)
    echo("</p>");
?>
```

PHP Outputs

`echo` and `print` statements are used to output data to the screen

echo: `echo(str);` or `echo str [,str...];`

- Can take multiple parameters
- Has no return value (i.e. cannot be used in expressions)

```
echo ($grade > 5) ? "pass" : "fail";
```

- Marginally faster than `print()`

print: `print(str);` or `print str;`

- Can take only one parameter
- Has a return value of 1 (i.e. can be used in expressions)

```
($grade > 5) ? print("pass") : print("fail");
```

Datatypes

Common datatypes supported by PHP:

- string (string values are enclosed in single or double quotes)
- int (32-bit or 64-bit number)
- float (platform-dependent but typically: 64-bit, with a precision of 14 decimal digits)
- bool (TRUE or FALSE values)

  0, 0.0, empty string, "0", NULL, empty array: FALSE

  Others: TRUE
- array (can store multiple values with different datatypes)
- object (the class of the object must be declared first)

Operators

- Arithmetic: +, –, *, /, %, ++, --, ** (exponentiation)
- Assignment: =, +=, -=, *=, /=
- Comparison: == (equal), === (identical), !=, <>, !==, >, <, >=, <=
- Logical: and, or, xor, &&, ||, !
- String: . (concatenation), .=
- Array: + (union), ==, ===, !=, !==, <>

Strings

String values are enclosed in single quotes, double quotes, or heredoc (starts with <<< followed by an identifier, ends with the identifier)

Example:

```
$str = <<<__ABC
This is a string
__ABC;
```

A string value can be expanded in multiple lines

Variables and escape sequences will not be expanded when using a single quote

Some string functions: `strlen()`, `strpos()`, `strrev()`,...

echo "<h2>$txt1</h2>"; vs echo '<h2>' . $txt1 . '</h2>';

Constants

A constant is an identifier (name) for a simple value

A valid constant name starts with a letter or underscore (no $ sign before the constant name)

Automatically global across the entire script

To create a constant, use the `define()` function

define(*name*, *value* [, *case-insensitive* = *FALSE*])

```
<?php
   define("GREETING", "Welcome to CICT!");
   echo GREETING;
?>
```

Control Statements

Condition statements:

- if … else
- switch … case
- ? :

Loops:

- while
- do … while
- for
- foreach

Control Statements – if ...else

```
if (condition1) {
   code to be executed if condition1 is true;
}
[ elseif (condition2) {
   code to be executed if condition2 is true;
} ... ]
[ else {
   code to be executed if all conditions are false;
} ]
```

```
date_default_timezone_set("Asia/Ho_Chi_Minh");
$t = date("H");
if ($t < "10") {
   echo "Have a good morning!";
} elseif ($t < "20") {
   echo "Have a good day!";
} else {
   echo "Have a good night!";
}
```

string date (string $format)

d - the day of the month (01 to 31)
m - Represents a month (01 to 12)
Y - Represents a year (in four digits)
l (lowercase 'L') - day of the week
H - 24-hour format of an hour
h - 12-hour format of an hour
i - Minutes with leading zeros
s - Seconds with leading zeros

Control Statements – switch … case

```
switch (n) {
  case label1:
    code to be executed if n=label1;
    break;
  case label2:
    code to be executed if n=label2;
    break;
  ...
  default:
    code to be executed if n
    is different from all labels;
}
```

```
switch ($dw) {
  case 'Saturday':
  case 'Sunday':
    echo "Nice weekend!";
    break;
  default:
    echo "Nice weekday!";
}
```

Control Statements – while

```
while (condition) {
    code to be executed;
}
```

```
$x = 1;
while ($x <= 5) {
  echo "The number is: $x <br>";
  $x++;
}
```

Control Statements – do … while

```
do {
    code to be executed;
} while (condition);
```

```
$x = 1;
do {
  echo "The number is: $x <br>";
  $x++;
} while ($x <= 5);
```

Control Statements – for/foreach

```
for (init counter; test counter; increment counter) {
    code to be executed;
}

foreach ($array as $value) {  //works only on arrays
    code to be executed;
}
```

```
for ($x = 0; $x <= 10; $x++) {
   echo "The number is: $x <br>";
}

$colors = array("red", "green", "blue", "yellow");
foreach ($colors as $value) {
   echo "$value <br>";
}
```

Content

1. Introduction
2. PHP language basics
3. PHP functions
4. OOP in PHP
5. PHP and forms
6. PHP and MySQL
7. Cookies and sessions
8. Advanced PHP techniques (file upload, pagination, AJAX, etc.)
9. Appendix

Declarations

A user-defined function is declared using the keyword `function`

Syntax:

```
function function_name() {
//function body (statements)
}
```

Note:

- A function name starts with a letter or underscore
- Function name should reflect what the function does
- Function names are NOT case-sensitive

Example

```
<html>
  <body>
    <?php
      function writeMsg() {
          echo "Hello world!";
      }

      writeMsg();
    ?>
  </body>
</html>
```

```
<!DOCTYPE html>
<html>
  <body>
    Hello world!
  </body>
</html>
```

Function Arguments

Declared inside the parentheses, separated by commas

```
<?php
  function ptb1($a, $b)
  {
      if ($a == 0) {
          if ($b == 0)
              echo "Infintely many solutions";
          else
              echo "No solution";
      }
      else
          echo "Solution: x = " . $a / $b;

  }
  ptb1(1, 2); //output: Solution x = 0.5
  ptb1(0, 2); //output: No solution
  ptb1(0, 0); //output: Infintely many solutions
?>
```

```
Solution: x = 0.5
No solution
Infintely many solutions
```

Function Arguments

May have default values (should be declared at the end)

```
<html>
  <body>
    <?php
      function createPar($noOfPar = 1, $idStart = 0) {
        for ($i=0; $i<$noOfPar; $i++) {
    ?>
          <p id='<?="par" . ($i+$idStart)?>'>... </p>
    <?php
        }  //for
      }  //createPar()

      createPar();
      createPar(2, 2);
    ?>
  </body>
</html>
```

```
<html>
  <body>
    <p id='par0'>...</p>
    <p id='par2'>...</p>
    <p id='par3'>...</p>
  </body>
</html>
```

Function Arguments

Functions arguments are pass-by-value by default

Pass-by-reference: prepend an & to the argument name

Example:

```
<?php
  function add_some_extra(&$string) {
    $string .= 'and something extra.';
  }

  $str = 'This is a string, ';
  add_some_extra($str);
  echo $str; //outputs 'This is a string, and something extra.'
?>
```

Function Arguments

Related built-in functions:

`int func_num_args(void)`: get the number of arguments

`mixed func_get_arg(int $arg_num)`: get the argument value

```php
<?php
  function foo() {
    $argsnum = func_num_args();
    echo "Number of arguments: $argsnum; ";
    echo "Argument values: ";
    for ($i = 0; $i<$argsnum; $i++)
      echo func_get_arg($i) . "\t";
  }
  foo();      //output: Number of arguments: 0; Argument values:
  foo(1, 2); //output: Number of arguments: 2; Argument values: 1    2
?>
```

Argument Type Declarations

Type declaration allows functions to require that parameters are of a certain type at call time (int, float, string, array, boolean,…)

To enable strict requirement: `declare(strict_types=1);`

```
<?php declare(strict_types=1); // strict requirement

  function addNumbers(int $a, int $b) {
      return $a + $b;
  }

  echo addNumbers(5, "5 days");
  //since strict is enabled and "5 days" is not an integer,
  //an error will be thrown
?>
```

```
Fatal error: Uncaught TypeError: Argument 2 passed to
addNumbers() must be of the type integer, string given,
```

Return Values

A function always returns one and only one value

To return a value for a function call: `return <value>;`

- Any type may be returned
- If no `return` statement in a function, `NULL` will be returned

```
<?php
  function square($num) {
      return $num * $num;
  }

  echo square(4);   // outputs '16'
?>
```

Return Type Declarations

Add a colon ":" and the type right before the "{" in the function declaration to specify the function return type

To enable strict requirements : `declare(strict_types=1);`

```
<?php declare(strict_types=1); // strict requirement
  function addNumbers($a, $b) : int {
       return $a + $b;
  }
  echo addNumbers(1, 1);
  echo addNumbers(1.2, 2.0);
?>
```

```
6
Fatal error: Uncaught TypeError: Return value of addNumbers
() must be of the type integer, float returned in /Users/tc
an/Desktop/code.php:3
```

Arrays

Arrays

An array stores multiple values in one single variable

Elements of an array may have different data types

Types of array:

- Indexed arrays: arrays with a numeric index
- Associative arrays: arrays with named keys
- Multidimensional arrays: arrays containing one or more arrays

Indexed Arrays

Declaration:

- Empty array: `$arr_name = array()`
- Array with initial values: `$arr_name = array(val1, val2,...);`

Access (set/get) array elements: `$arr_name[index]`

Get array length: `count($arr_name)`

Loop through an array:

- `for` loop (usually combined with the `count()` function)
- `foreach ($arr_name as $var)` loop

✔ Array index may not be contiguous

✔ Index may be omitted, $a[]: a new available index will be used

Indexed Arrays

```php
<?php
  $cars = array("Volvo", "BMW", "Toyota");
  $arrlength = count($cars);

  for($x = 0; $x < $arrlength; $x++) {
      echo $cars[$x];
      echo "<br>\n";
  }
?>
```

```
Volvo<br>
BMW<br>
Toyota<br>
```

- Unset($cars[1])
- Array_splice($cars, 1, 1)

Associative Arrays

Two ways to create an associative array:

```
$arr_name = array(key => value [, key => value ...] );
$arr_name['key1'] = value1;
$arr_name['key2'] = value2;
```

Access array elements: `$arr_name['key']`

Get array key set: `array_keys($arr_name)`

Loop through an associative array:

- `foreach ($arr_name as $key => $value) { ... }`
- `for` loop using the key set

Remove a key/value pair: using `unset($arr[key])` function

Associative Array Example

```php
<?php
  $age = array("Peter"=>"35", "Ben"=>"37", "Joe"=>"43");
  foreach ($age as $x => $x_value) {
    echo "Key=" . $x . ", Value=" . $x_value;
    echo "<br>";
  }
?>
```

```php
<?php
  //another method to loop through an associative array
  $keys = array_keys($age);
  for ($i=0; $i<count($keys); $i++) {
    echo $keys[$i] . ": " . $age[$keys[$i]] . "\n";
  }
?>
```

```
Key=Peter, Value=35
Key=Ben, Value=37
Key=Joe, Value=43
```

Multi-dimensional Arrays

A multi-dimensional array is an array of arrays

- Two dimensional: an array of arrays
- Three dimensional: an array of arrays of arrays

Syntax:
```
$arr_name = array(
array(...),
array(...)
[,...]);
```

Arrays of an array may be different dimensions

Multi-dimensional Arrays

```php
<?php
  $cars = array (
    array("Volvo",22,18), array("BMW",15,13),
    array("Saab",5,2), array("Land Rover",17,15));

  for ($row = 0; $row < 4; $row++) {
    echo "<p><b>Row number $row</b></p>";
    echo "<ul>";
    for ($col = 0; $col < 3; $col++) {
      echo "<li>".$cars[$row][$col]."</li>";
    }
    echo "</ul>";
  }
?>
```

Row number 0

- Volvo
- 22
- 18

Row number 1

- BMW
- 15
- 13

Row number 2

- Saab
- 5
- 2

Row number 3

- Land Rover
- 17
- 15

Multi-dimensional Arrays

```php
<?php
  $fruits = array(
     "fruits" => array("a"=>"orange", "b"=>"banana",
                       "c"=>"apple"),
     "numbers" => array(1, 2, 3, 4));
  // Some examples to address values in the array above
  echo $fruits["fruits"]["a"]; //prints "orange"
  echo $fruits["numbers"][2];  //prints "3"
  // Add a new element to $fruits
  $fruits["apple"]["green"] = "good";
  var_dump($fruits);
?>
```

```
array(3) {
  ["fruits"]=>
  array(3) {
    ["a"]=>
    string(6) "orange"
    ["b"]=>
    string(6) "banana"
    ["c"]=>
    string(5) "apple"
  }
  ["numbers"]=>
  array(4) {
    [0]=>
    int(1)
    [1]=>
    int(2)
    [2]=>
    int(3)
    [3]=>
    int(4)
  }
  ["apple"]=>
  array(1) {
    ["green"]=>
    string(4) "good"
  }
}
```

Content

1. Introduction
2. PHP language basics
3. PHP functions
4. OOP in PHP
5. PHP and forms
6. PHP and MySQL
7. Cookies and sessions
8. Advanced PHP techniques (file upload, pagination, AJAX, etc.)
9. Appendix

Class Definition

Syntax:

```php
class SimpleClass {
    //property declaration: similar to variable declaration
    public $var = 'a default value';

    //method declaration: similar to function declaration
    public function displayVar() {
        echo $this->var;
    }
}
```

Access modifiers: `public`, `protected`, `private`

`$this`: reference to the calling object

Creating Objects

Use the new keyword: `$var = new Classname([arg]);`

If there is no argument for the constructor, parentheses may be omitted

```php
<?php
  $obj1 = new SimpleClass();

  //This can also be done with a variable:
  $className = 'SimpleClass';
  $obj2 = new $className(); // new SimpleClass()


  $obj1->displayVar();
?>
```

Object Assignment

`$obj2 = $obj1`: $obj2 references to the same object as $obj1 (shadow copy)

`$obj2 = &$obj1`: $obj2 is a reference of $obj1 (reference is an alias)

`$obj2 = clone $obj1`: A new object is cloned from $obj1 and it is referenced by $obj2 (deep copy)

```php
<?php
  $obj1 = new SimpleClass();

  $obj2 = $obj1;
  $obj3 = clone $obj1;
  $obj4 = &$obj1;

  $obj1->var = "new val";

  $obj1->displayVar(); //new val
  $obj2->displayVar(); //new val
  $obj3->displayVar(); //default
  $obj4->displayVar(); //new val

  $obj1 = null;

  var_dump($obj1); //NULL
  var_dump($obj2); //new val
  var_dump($obj3); //default
  var_dump($obj4); //NULL
?>
```

Static Members

Declared using `static` keyword

Accesses though class, not the objects of the class

Outside the class:

`classname::$property`

`classname::method()`

Inside class:

`self::$property`

`self::method()`

Note: `$this` cannot be used inside static methods

```php
<?php
  class Foo {
      public static $static_v = 'foo';

      public function staticValue() {
          return self::$static_v;
      }
  }

  print Foo::$static_v . "\n";

  $foo = new Foo();
  print $foo->staticValue() . "\n";
?>
```

Constructors and Destructors

Constructors: called automatically when an object is created

```
public function __construct($arg1, $arg2,...) {
   //initialization
 }
```

Destructors: called automatically when an object is destroyed

```
public function __destruct($arg1, $arg2,...) {
   //clean-up tasks
 }
```

Constructors and Destructors

```php
<?php
  class Person {
    public $id;  public $name;
    public $dob;
    protected static $count = 0;

    public function __construct($name) {
      $this->id = ++self::$count;
      $this->name = $name;
      $this->dob = date("d/m/Y");
    }
    public function __destruct() {
      $this->id = --self::$count;
    }
    public function displayInfo() {
      echo $this->id . " - "
          . $this->name . " - "
          . $this->dob . " ("
          . self::$count . ")";
    }
  }
?>
```

```
1 – Mr. Tom – 27/03/2019 (1)
2 – Ms. Jerry – 01/01/2002 (2)
1 – Mr. Tom – 27/03/2019 (1)
```

```php
<?php
  $tom = new Person("Mr. Tom");
  $tom->displayInfo();
  //output: 1 - Mr. Tom - 07/03/2014
  $jerry = new Person("Ms. Jerry");
  $jerry->dob = "01/01/2002";
  $jerry->displayInfo();
  //output: 2 - Ms. Jerry - 01/01/2002
  unset($jerry);
  $tom->displayInfo();
?>
```

Inheritance

Syntax: use the `extends` keyword

```
class classname extends parentClassname {
    //child class members
}
```

Child class inherits all members of the parent class (but it can access public and protected members)

Child class can override methods of the parent class

Access parent members from the child class:

```
parent::property
parent::method()
```

Inheritance

```php
<?php
  class Student extends Person {
    public $enroll;

    function __construct($name) {
      parent::__construct($name);
      $this->enroll = @date("d/m/Y");
    }

    public function displayInfo() {
      parent::displayInfo();
      echo " - " . $this->enroll;
    }
  }
?>
```

```php
<?php
  $tom = new Person("Mr. Tom");
  $tom->displayInfo();
  //output: 1 - Mr. Tom - 07/03/2014
  $jerry = new Person("Ms. Jerry");
  $jerry->enroll = "01/01/2002";
  $jerry->displayInfo();
  //output: 2 - Ms. Jerry -
07/03/2014 - 01/01/2002
?>
```

Further Reading

- Abstract class (class contains some abstract methods – methods without implementation)
- Final methods (methods that cannot be overridden)
- Interfaces ("pure" abstract classes)
- `__toString()` method (object to string auto conversion)
- Using code from other .php files (`include` statement)

Content

1. Introduction
2. PHP language basics
3. PHP functions
4. OOP in PHP
5. PHP and forms
6. PHP and MySQL
7. Cookies and sessions
8. Advanced PHP techniques (file upload, pagination, AJAX, etc.)
9. Appendix

HTML Forms

Forms are used to get user inputs

Form data is usually sent to the server (PHP) to process

```
<form action="..." method="...">
    <!-- form controls -->
</form>
```

action: PHP page that will receive and process form data

method: HTTP method that is used to send data to the server

GET: form data is sent via the URL parameters (1K-4KB)

POST: form data is sent inside the request package body

Form data is sent to the server in the form of an array (key => value) where key is the name of the control and value is the input data

Getting Form Data

The PHP global variables `$_GET` and `$_POST` are used to collect form data (corresponding to the GET and POST method)

```
<!- welcome.html -->
<html>
  <body>
    <form action="welcome.php" method="get">
      Name: <input type="text" name="name"><br>
      E-mail: <input type="text" name="email"><br>
      <input type="submit">
    </form>
  </body>
</html>
```

Name: An
E-mail: tcan@cit.ctu.edu.vn
Submit

```
<!- welcome.php -->
<html>
  <body>
    Welcome <?php echo $_GET["name"]; ?><br>
    Your email is: <?php echo $_GET["email"]; ?>
  </body>
</html>
```

Welcome An
Your email is: tcan@cit.ctu.edu.vn

Checking Data Existence

Use the `isset()` function:

```
<body>
   <h3>
     <?php
       if (isset($_POST["uname"]) && isset($_POST["pwd"])) {
         if ($_POST["pwd"] == "abc") {
           echo "Welcome " . $_POST["uname"];
         } else {
           echo "Sorry " . $_POST['uname'] . ", wrong pass!";
         }

       } else {
         echo "<span style=\"color: red;\">"
              . "<i>uname</i> and <i>pword</i> are expected</span>";
       }
       ?>
   </h3>
  </body>
```

Content

1. Introduction
2. PHP language basics
3. PHP functions
4. OOP in PHP
5. PHP and forms
6. PHP and MySQL
7. Cookies and sessions
8. Advanced PHP techniques (file upload, pagination, AJAX, etc.)
9. Appendix

MySQL Server Access from PHP

Two methods in accessing MySQL DB from PHP:

- **MySQLi** (object-oriented or procedural, only work with MySQL)
- PDO (PHP Data Objects, can work with 12 DBMS)

Steps in accessing MySQL databases from PHP:

1. Create connection to MySQL server
2. Select the database
3. Execute SQL statement (query, insert, update, delete)
4. Get and process the returned data
5. Produce the output
6. Close the connection

Related MySQLi Classes

`mysqli`: represents a connection between PHP and MySQL

- `::__construct()`: creates connections
- `::$connect_errno`: contains the last connection error code (0: no error)
- `::$connect_error`: contains the last connection error description
- `::select_db()`: select the default DB
- `::query()`: perform a query
- `::$errno`: error code for the most recent function call (0: no error)
- `::$error`: contains the last error description
- `::prepare()`: create a prepared statement (`mysqli_stmt` class)
- `::close()`: close a connection

Related MySQLi Classes

`mysqli_result`: represents the result set obtained from a query

- `::fetch_array()`: afetch a row as an associative or numeric array or both
- `::fetch_assoc()`: fetch a row as an associative array
- `::fetch_all()`: fetch all rows as an associative or numeric array or both
- `::fetch_row()`: fetch a row as an numeric array
- `::$field_count`: get the number of fields in a result
- `::$num_rows`: get the number of rows in a result

Creating Connections

Use the constructor of the class `mysqli`:

```
mysqli($servername, $username [, $password, $dbname]);
```

- servername: can be either hostname or an IP
- username: MySQL user name
- password: MySQL user password (not provided/NULL: no password)
- returns an object representing the connection to MySQL server

The connection information:

- usually declared in a separated file
- included into the PHP file using the `require_once()` statement

Creating Connections

```php
<?php
  require_once 'connection.inc';

  //create connection
  $conn = @new mysqli($servername, $username, $password);

  //check connection
  if ($conn->connect_error) {
      die("Connection failed: " . $conn->connect_error);
  }
  echo "Connected successfully";
  ?>
```

```php
<?php   /* connection.inc */
  $servername = "localhost";
  $username = "root";
  $password = "1234";
?>
```

`mysqli::$connect_error` – contains description of the last error, or NULL of no error occurred

Selecting DB

Select DB when create the connection:

`mysqli($servername, $username, $password, $dbname);`

Select DB after the connection created:

`mysqli::select_db($dbname)`: returns TRUE on success, FALSE on failure

```
<?php
  $db = "test";
  if ($conn->select_db($db)) {  //$conn: connection object
    echo "Select DB '$db' successfully";
  }
  else {
    echo "Couldn't select the '$db' DB";
  }
?>
```

Executing SQL Statements

Use the `mysqli::query()` method:

- Failure: returns FALSE
- Success: `mysqli_result` object for SELECT, SHOW, DESCRIBE, EXPLAIN queries; TRUE for other queries

```php
<?php
  $sql = "SELECT id, firstname, lastname FROM students";
  $result = $conn->query($sql);
  if ($result && $result->num_rows > 0) {
      // process the received data
  } else {
      // query failed OR no data received
  }
?>
```

Accessing Returned Data

Access data in a `mysqli_result` object with `fetch_assoc()`:

```
<?php
  //...
  $result = $conn->query($sql);

  if ($result->num_rows > 0) {
    // output data of each row
    while($row = $result->fetch_assoc()) {
      echo "ID: " . $row["id"] . " - Name: "
           . $row["firstname"]. " " . $row["lastname"]. "<br>";
    }
  } else {
      echo "No student found";
  }

  $conn->close();
?>
```

Accessing Returned Data

Access data in a `mysqli_result` object with `fetch_row()`:

```
<?php
  //...
  $result = $conn->query($sql);

  if ($result->num_rows > 0) {
    // output data of each row
    while($row = $result->fetch_row()) {
      echo "ID: " . $row[0]. " - Name: "
           . $row[1]. " " . $row[2]. "<br>";
    }
  } else {
      echo "No student found";
  }

  $conn->close();
?>
```

Accessing Returned Data

Access data in a `mysqli_result` object with `fetch_array()`:

```
<?php
  //...
  $result = $conn->query($sql);

  if ($result->num_rows > 0) {
    // output data of each row
    while($row = $result->fetch_array(MYSQLI_BOTH)) {
      echo "ID: " . $row[0]. " - Name: "
              . $row["firstname"]. " " . $row["lastname"]. "<br>";
    }
  } else {
      echo "No student found";
  }

  $conn->close();
?>
```

Display error messages

Two ways to display error messages when executing a PHP script

Add the following code:

```
ini_set('display_errors', 1);
```

Or change the configuration in the PHP.ini file with the following directive:

```
display_errors=on
```

SQL Injection

It is a code injection technique that can destroy databases.

It is one of the most common web hacking techniques.

It involves placing malicious code into SQL statements through website input.

Some common SQL Injection techniques include:

- SQL Injection based on 1=1 (always true)
- SQL Injection based on ""="" (always true)”
- SQL Injection based on Batch SQL Statements

SQL Injection: 1=1 (always true)

Example:

To retrieve user information corresponding to a specific userId entered by the user from an HTML form.

```
$txtUserId = $_POST["UserId"];
$txtSQL = "SELECT * FROM Users WHERE UserId = ".$txtUserId;
```

However, the user can enter the following string: 105 OR 1=1

The result is that the following SQL statement will be executed on the system:

```
SELECT * FROM Users WHERE UserId = 105 OR 1=1;
```

This statement is valid and will retrieve all data from the Users table. If the Users table contains sensitive information such as usernames and passwords, this could lead to a data leak.

SQL Injection: ""="" (always true)

Example:

To retrieve user information based on a condition formed from two parameters entered by the user from the HTML form

```
$uName = $_POST["username"];
$uPass = $_POST["userpassword"];
$sql = 'SELECT * FROM Users WHERE Name ="'.$uName.'" AND Pass ="'.$uPass.'"'
```

However, users can enter the following string for both the username and password fields in the HTML form : " or ""="

The result is that the following SQL statement will execute :

```
SELECT * FROM Users WHERE Name ="" or ""="" AND Pass ="" or ""=""
```

This statement is valid and will retrieve all data from the Users table

SQL Injection: batch SQL statements

Most databases support executing multiple SQL statements on the same line.

Example:

```
$txtUserId = $_POST["UserId"];
$txtSQL = "SELECT * FROM Users WHERE UserId = " + $txtUserId;
```

The user can enter the following string: 105; DROP TABLE Suppliers

The result is that the following SQL statement will be exec

```
SELECT * FROM Users WHERE UserId = 105; DROP TABLE Suppliers;
```

This statement is valid and will delete the Suppliers table

Prevent SQL Injection

Using Prepared statements:

- Supported by both MySQLi and PDO

Executing SQL Statements

Prepared statements:

`mysqli::prepare($query)`: create prepared statement

- Returns a `mysqli_stmt` object or FALSE if an error occurred

`mysqli_stmt::bind_param($type, $var [, $var…])`: binds variables to a prepared statement as parameters

- Returns TRUE on success; FALSE on failure
- Parameter types: i (integer), d (double), s (string), b (blob)

`mysqli_stmt::execute()`: execute a prepared query

- Returns TRUE on success; FALSE on failure

Prepared statement is recommended due to its better performance, less bandwidth and useful against SQL injection

Related MySQLi Classes

`mysqli_stmt`: represents a prepared statement

- `::bind_param()`: binds variables to a prepared statement as parameters
- `::bind_result()`: binds variables to a prepared statement for result storage
- `::execute()`: executes a prepared statement
- `::get_result()`: returns the result set from the statement
- `::fetch()`: fetch results from a prepared statement
- `::$num_rows`: number of rows in the statement result set
- `::close()`: closes a prepared statement

Prepared statements

Used to repeatedly execute the same (or similar) SQL statements

Work as follows:

1. Preparation: A sample SQL statement is created and sent to the database. Certain values are not specified (labeled "?").

   For example: `INSERT INTO MyGuests VALUES(?, ?, ?)`

1. The database parses, compiles, and optimizes the query on the sample SQL statement and stores the result without executing it.
2. Execution: The application then associates the values with the parameters, and the database executes the statement. The application can execute the statement as many times as it likes with different values.

Prepared statements

Compared to directly executing SQL statements, prepared statements offer three main advantages:

1. reduce parsing time because query preparation is only performed once
2. parameters binding minimizes bandwidth to the server because you only need to send the parameters each time, not the entire query.
3. useful in preventing SQL injection, because the parameter values are passed later using a different protocol, which doesn't need to be "escaped" precisely

Prepared statements

Syntax for using Prepared statements in PHP with MySQLi:

Certain values are left unspecified, called parameters (labeled "?"). Example:

"INSERT INTO MyGuests (firstname, lastname, email) VALUES (?, ?, ?)"

Using bind_param() to bind the parameters to the prepared SQL statement. Example:

```
$stmt->bind_param("sss", $firstname, $lastname, $email);
```

The "sss" argument lists the data types containing the parameters. The character 's' tells MySQL that the parameter is a string. Types:

i - integer

d - double

s - string

b - BLOB

Executing SQL Statements

```
<?php
  // prepare and bind
  $stmt = $conn->prepare("INSERT INTO courses VALUES (?, ?, ?)");
  $stmt->bind_param("ssi", $courseId, $courseName, $credits);

  // set parameters and execute
  $courseId= "CT214H";
  $courseName = "Web Programming";
  $credits = 3;
  $stmt->execute();   //insert the 1st record

  $courseId= "CT108H";
  $courseName = "Object-Oriented Pgrogramming";
  $credits = 3;
  $stmt->execute();   //insert the 2nd record

  $stmt->close(); //close the prepared statement
  $conn->close(); //close the connection
<?php
```

Accessing Returned Data

Access data in a `mysqli_stmt` object with `bind_result()` and `fetch()`:

```
<?php
  //...
  if ($stmt = $mysqli->prepare("SELECT id, firstname, lastname
                                FROM students")) {
    $stmt->execute();

    // bind variables to prepared statement
    $stmt->bind_result($col1, $col2, $col3);

    while ($stmt->fetch()) {   //fetch values
      printf("%s %s %s\n", $col1, $col2, $col3);
    }
    $stmt->close();
  }
  $mysqli->close();
  ?>
```

Template for PHP-MySQL Data Access

```php
<?php /* php-mysql-template-procedural.php */
  //connect + select DB
  $conn = @mysqli_connect($hostname, $username, $pwd, $db) or die("...");
  //create query string + execute query
  //OR use prepared statement for better performance
  $query = "SELECT * FROM table_name";
  $result = @mysqli_query($conn, $query) or die("...");

  //process the returned data
  while ($row = mysqli_fetch_assoc($result)) {
    //...process the record...
  }
  mysqli_close();
```

Procedural

Template for PHP-MySQL Data Access

```php
<?php mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
    try { //connect + select DB
      $conn = new mysqli("127.0.0.1", "root", "", "classics");
    } catch (Exception $e) {
      die ("Connection failed: " . $e->getMessage() . "\n");
    }
    echo "Connected!\n";

    try { //create query string + execute query
      $query = "SELECT * FROM classics";
      $result = $conn->query($query);
    } catch (Exception $e) {
      die ("Query failed: " . $e->getMessage() . "\n");
    }
    echo "Query successfully!\n";
```

OOP Exception

Content

1. Introduction
2. PHP language basics
3. PHP functions
4. OOP in PHP
5. PHP and forms
6. PHP and MySQL
7. **Cookies and sessions**
8. Advanced PHP techniques (file upload, pagination, AJAX, etc.)
9. Appendix

What is Cookie?

Cookies are data, stored in a small text file on the browser (user computer) by the server

This small file stores a set of `name => value` items (called cookie)

Usually used to identify the user

Each time the browser making a request, it will include the cookies in the request message

To create a cookie (name => value) in PHP:

```
setcookie(name [, value, expire ...])
```

Expiration unit is second

The calls to this function must appear before the `<html>` tag

Getting and Deleting Cookies

Cookies sent to server is stored in the array variable `$_COOKIE`

To get a cookie value, provide cookie name `$_COOKIE[name]`

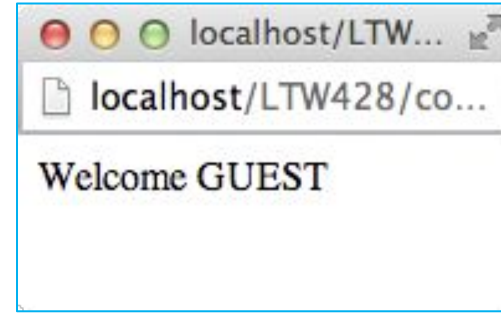
A cookie is automatically deleted when:

- Time is expire
- The browser closes, if no expire is set

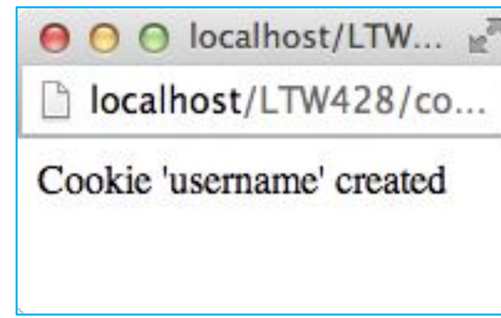
To delete a cookie, set expire time of the cookie with an expiration date in the past

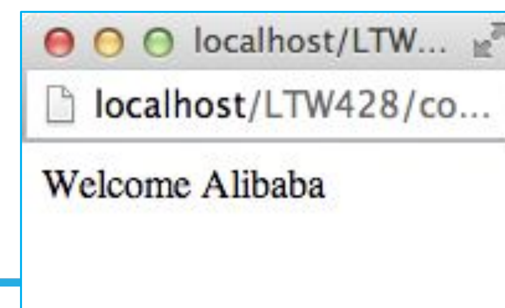
Cookie Example

```php
<html>
  <body>
    <?php
      if (isset($_COOKIE['username']))
         echo "Welcome $_COOKIE[username]";
      else
         echo "Welcome GUEST";
      ?>
  </body>
</html>
```




```php
<?php
  //expiration: 600 seconds
  setcookie("username", "Alibaba",
            time() + 60 * 10);
?>
<html>
  <body>
    <p>Cookie 'username' created</p>
  </body>
</html>
```





Sessions

Used to store information (in variables) across multiple pages of an application (website)

Sessions are stored on the server

By default, session variables last until the browser closes

To start a session: call `session_start()` (before the `<html>` tag)

Session variables are stored in the global variable `$_SESSION`

- Access a session: `$_SESSION[name]`

Destroy all sessions: `session_unset()` or `session_destroy()`

Destroy a session variable: `unset($_SESSION[name])`

Session Example – Login

Sessions

```
<?php    /* login.php */
  //$login_ok = /* check the login information */;
  if ($login_ok) {
     session_start();
     $_SESSION['logged_in'] = true; //OR = username
     /* redirect to other page */
  }
?>

<html>
 <!-- login form -->
</html>


<?php    /* logout.php */
  session_start();
  if (isset($_SESSION['logged_in']))
     unset($_SESSION['logged_in']);
?>
```

Session Example – Login

```
<?php    /* update-data.php */
  session_start();
  if (!isset($_SESSION['logged_in'])) {
    header('Location: login-session.php');
    die();
  }
?>

<html>
  <!-- update data form... -->
</html>
```

The code to check login session is usually stored in a separated file and included into the webpages which need the authorization

Content

1. Introduction
2. PHP language basics
3. PHP functions
4. OOP in PHP
5. PHP and forms
6. PHP and MySQL
7. Cookies and sessions
8. Advanced PHP techniques (file upload, pagination, AJAX, etc.)
9. Appendix

File Upload

Setting Up

Turn on file upload option:

Set the `file_uploads` directive to On in file “**php.ini**”

XAMPP:

Windows: [XAMPP_HOME]\php\php.ini

Steps to upload files by PHP:

1. Create a form for file upload with appropriate encryption method (browser)
2. Validate data received at server (error, format, size, etc.)
3. Save files received to storage devices

Upload Form (Client/Browser)

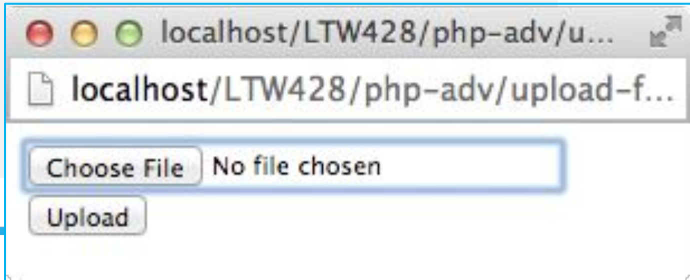
Form attributes:

Method: POST

Encryption method: enctype=multipart/form-data

File select control: <input type=file ...>

```html
<html> <!-- upload-form.html -->
  <body>
    <form method="POST" action="upload.php"
          enctype="multipart/form-data">
      <input type="file" name="up-file"><br>
      <input type="submit" value="Upload">
    </form>
  <body>
</html>
```



File Information (Server)

After uploading, the file will be stored temporarily on the server (configured with the `upload_tmp_dir` directive in **php.ini**)

The global associative array `$_FILES` contain all the uploaded file information:

- `$_FILES["filename"]["name"]`: the original name of the file
- `$_FILES["filename"]["type"]`: the MIME type of the file
- `$_FILES["filename"]["size"]`: the file size (in bytes)
- `$_FILES["filename"]["tmp_name"]`: the temporary name of the file (stored temporarily in the server after uploading)
- `$_FILES["filename"]["error"]`: error code of the upload

Validation (Server)

File Upload

```
<?php    /* upload.php (pre-processing) */
 if (($_FILES['up-file']['error'] == 0) &&   //upload error?
     ($_FILES["up-file"]["type"] == "...") &&  //expected type?
     ($_FILES["up-file"]["size"] < ...))  //size excess?
       //code for saving file
 }
    else {
       //error handler ...
       echo "Upload error: " . $_FILES['up-file']['error'];
 }
?>
```

File Storage (Server)

Related functions:

`move_uploaded_file(tmp_file, persistent_file)`: moves an uploaded file to a new location (returns TRUE on success)

`file_exists(filename)`: check whether a file or directory exists (returns TRUE on success)

```
<?php

  // verify file upload conditions (pre-processing) ...

  // save file to folder “uploads”
  move_uploaded_file($_FILES["up-file"]["tmp_name"],
          "uploads/" . $_FILES["up-file"]["name"]);

 // post processing ...

?>
```

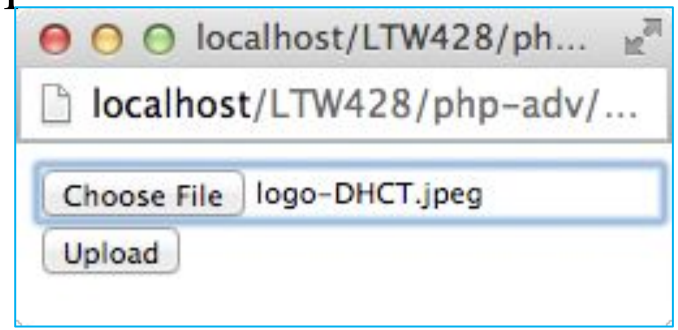
Complete Image Upload Example

1. Check for upload error and file type (jpeg)
2. Save image file to "uploads" folder
3. Display uploaded image on upload page

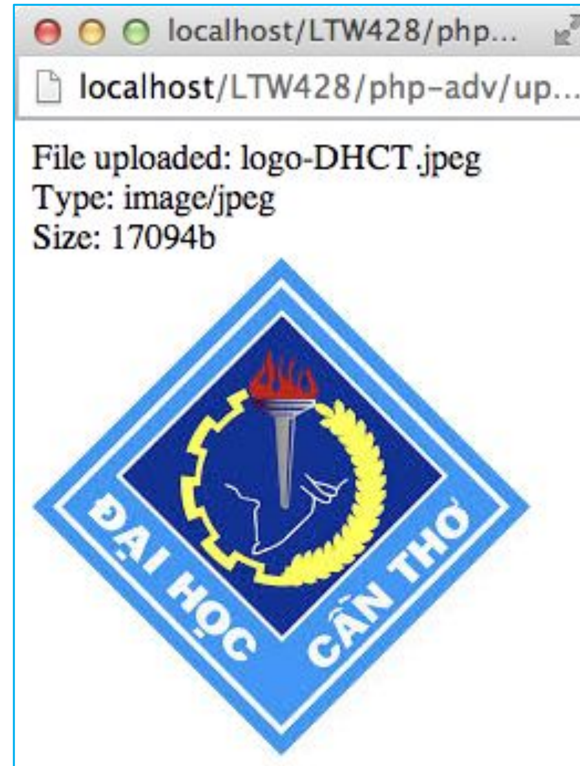
```
<?php   /* upload.php */
  if (($_FILES['up-file']['error'] == 0)) {

        move_uploaded_file($_FILES["up-file"]["tmp_name"],
          "uploads/" . $_FILES["up-file"]["name"]);

        echo "File uploaded: " . $_FILES["up-file"]["name"] . "<br>";
        echo "Type: " . $_FILES['up-file']['type'] . "<br>";
        echo "Size: " . $_FILES['up-file']['size'] . "b<br>";
        echo '<img src="uploads/' . $_FILES["up-file"]["name"] . '"/>';
  }
  else
     echo "Upload error: " . $_FILES['up-file']['error'];
?>
```

Complete Image Upload Example

File Upload



upload-form.html



upload.php

Saving Images to a Database

Datatype to store image: `blob`

Steps to save image to database:

1. Validate file uploaded: upload status, type, size, etc. (using the global variable `$_FILES` and function `getimagesize()`)
2. Get the image file content which is saved temporarily on the server to a variable
3. Add backslashes (\ ) before predefined character, using the `addslashes()` function
4. Save image into the table, using the image variable create in step 2

Saving Images to a Database

Given a table with the following structure:

```
MariaDB [ct219h]> describe images;
+---------+-----------+------+-----+---------+
| Field   | Type      | Null | Key | Default |
+---------+-----------+------+-----+---------+
| imgname | char(100) | NO   | PRI | NULL    |
| imgdata | longblob  | NO   |     | NULL    |
+---------+-----------+------+-----+---------+
```

Upload form:

```
<form method="POST" action="upload-img-db.php"
    enctype="multipart/form-data">

  <input type="file" name="img_file">
  <input type="submit" value="Upload">
</form>
```

Saving Images to a Database

```php
<?php /* upload-img-db.php */

 //connect to DBMS + select DB... ($conn: connection object)

 if (isset($_FILES["img_file"]["name"]) &&
     getimagesize($_FILES['img_file']['tmp_name']) != false) {

   //CHECK other CONTRAINTS if any ...
   $image = file_get_contents($_FILES['img_file']['tmp_name']);
   $image = addslashes($image);
   $imgname = $_FILES["img_file"]["name"];
   $conn->query("INSERT INTO images VALUES ('$imgname', '$image')")
      or die("Cannot insert image into DB: " . $conn->connect_error);

   echo "Uploaded image: <br><img src='get_img.php?name=$imgname'>";
 }
 else
   echo "No image has been uploaded";
?>
```

Displaying Images Saved in a Database

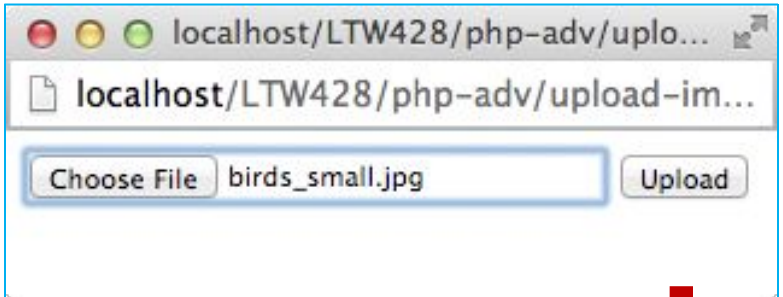
To display an image in DB: `<img src=get_img?id=img_id>`

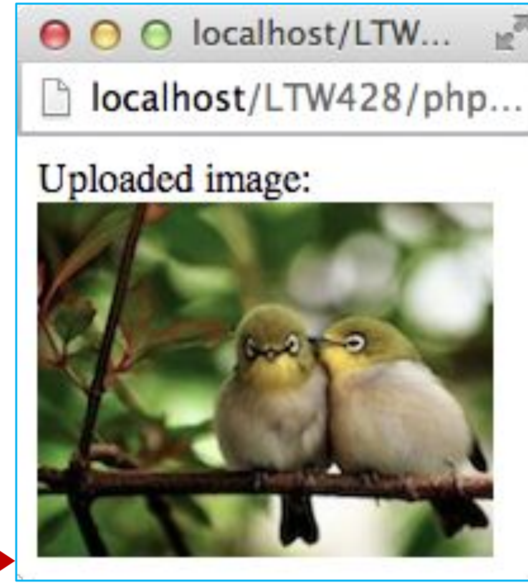
(***get_img.php*** *is script that read the image with the* ***id*** *is* ***img_id*** *from the DB*)

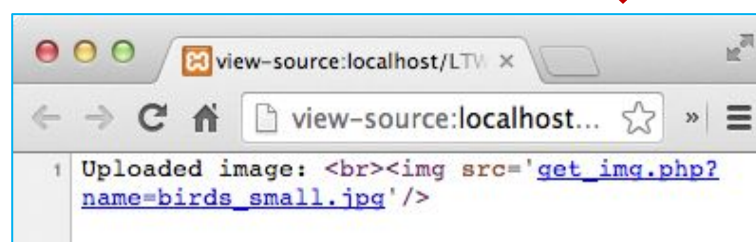
```
<?php   /* get_img.php */
 if (isset($_REQUEST['name'])) {
   // connect to DBMS and select the DB... ($conn: connection object)
   $id = $_GET['id'];
   $stmt = $conn->prepare("SELECT imgdata FROM images WHERE imgname = ?");
   $stmt->bind_param("s", $id);
   $stmt->execute();
   $stmt->bind_result($imgdata);
   $stmt->fetch();

   header("Content-Type: image/jpeg");
   echo $imgdata;
   $stmt->close();

?>
```

Displaying Images Saved in a Database

File Upload







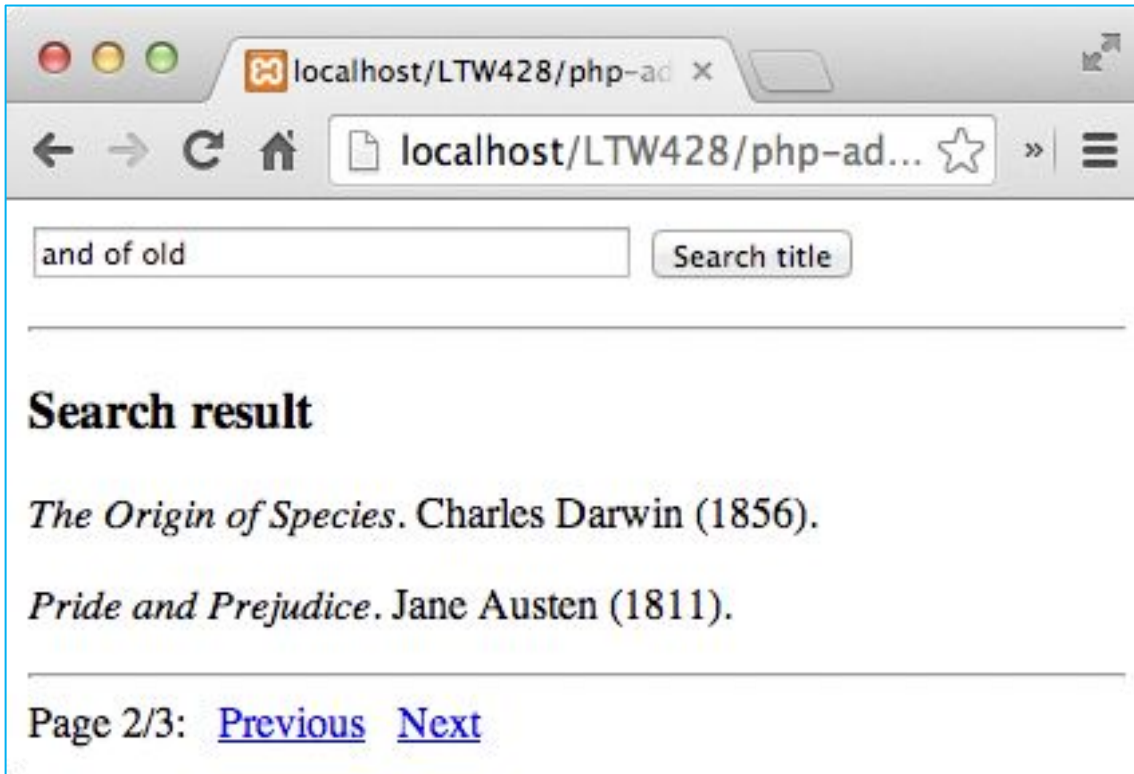
Pagination

Steps in Creating Pagination

1. Calculate the total number of records: `$total_records`
2. Identify the number of records displayed per page: `$records_per_page`
3. Store the current page number (use hidden variable or send directly to server by GET method)
4. Query data for the requested page
5. Create Next page and Previous page links

Case Study

Build a book search page with pagination (2 results per page)



Structure of the Application

Main page:

Show the form to get search keywords + Search button

Call `search()` function to search for books and get the pagination information

Call `page_nav_link()` function to create links to Prev/Next page

PHP functions:

`compute_paging(search_kw)`: computer pagination parameters

`search(kw)`: search and return search result and pagination info

`page_nav_links(paging_info, search_kw)`: create links to Prev/Next page

Main page

```
<html>
 <body>
  <!-- title-search-paging.php -->
  <form action="title-search-paging.php" method="GET">
   <input type="text" size="40" name="search_kw"
    value="<?php empty($_REQUEST['search_kw']) ||
     print $_REQUEST['search_kw'];?>"/>
   <input type="submit" value="Search title">
  </form>
  <hr>
  <h3>Search result</h3>
  <?php
   if (isset($_REQUEST['search_kw'])) {
    include 'title-search-paging-func.php';
    $paging = search($_REQUEST['search_kw']);
    echo "<br><hr>";    //links to next/prior page
    page_nav_links($paging, $_REQUEST['search_kw']);
   }
   ?>
 </body>
</html>
```

Function compute_paging()

```php
<?php   /* title-search-paging-func.php */
 $record_ppage = 2;

 function compute_paging($search_kw) {
   global $record_ppage;
   $query = "SELECT count(*) FROM classics "
            . "WHERE title LIKE '%$search_kw%'";
   $result = $conn->query($query);
   $row = $result->fetch_row();
   $p_total = ceil($row[0]/$record_ppage);
   $page = (isset($_REQUEST["page"]))? $_REQUEST["page"] : 1;
   $start = ($page - 1) * $record_ppage;
   $p_prev = ($page > 1)? $page - 1 : 0;
   $p_next = ($page < $p_total)? $page + 1 : 0;
   return array("p_total"=>$p_total, "p_no"=>$page,
             "p_start"=>$start, "p_prev"=>$p_prev,
             "p_next"=>$p_next, "total"=>$row[0]);
 } //compute_paging()
```

p_total: total no of pages;
p_no: current page;
p_start: index of the first record in the page;
total: total number of records

Function search()

```
<?php /* title-search-paging-func.php */
 function search($keyword) {
  global $record_ppage;
  $search_kw = str_replace(" ", "%' OR title LIKE '%", trim($keyword));

  $paging = compute_paging($search_kw);

  $query = "SELECT * FROM classics WHERE title LIKE '%$search_kw%'" .
           " LIMIT $paging[p_start], $record_ppage";

  $result = $conn->query($query)
  or die ("DB accessed failed: " . $conn->error);

  while ($row = $result->fetch_assoc())
    echo "<p><i>$row[title]</i>. $row[author] ($row[year]).";

  if ($result->num_rows == 0)
    echo "No title found";
  return $paging;
 } //search()
?>
```

$paging	
	p_total: total no of pages; **p_no**: current page; **p_start**: index of the first record in the page; **total**: total number of records

Function search()

F

```
<?php      /* title-search-paging-func.php */
 function page_nav_links($paging, $search_kw) {
  echo "Page $paging[p_no]/$paging[p_total]:   ";

  if ($paging['p_prev'] > 0) {    //previous
   echo "<a href='title-search-paging.php?search_kw=$search_kw" .
    "&page=" . $paging['p_prev'] ."'>Previous</a>   ";
  }

  if ($paging['p_next'] > 0) {    //next
   echo "<a href='title-search-paging.php?search_kw=$search_kw" .
    "&page=" . $paging['p_next'] ."'>Next</a>";
  }
 } //page_nav_links()
?>
```

AJAX *Asynchronous JavaScript And XML*

What is AJAX?

A technology that allow asynchronous communication between web browsers and web servers

Not a programming language, just a combination of:

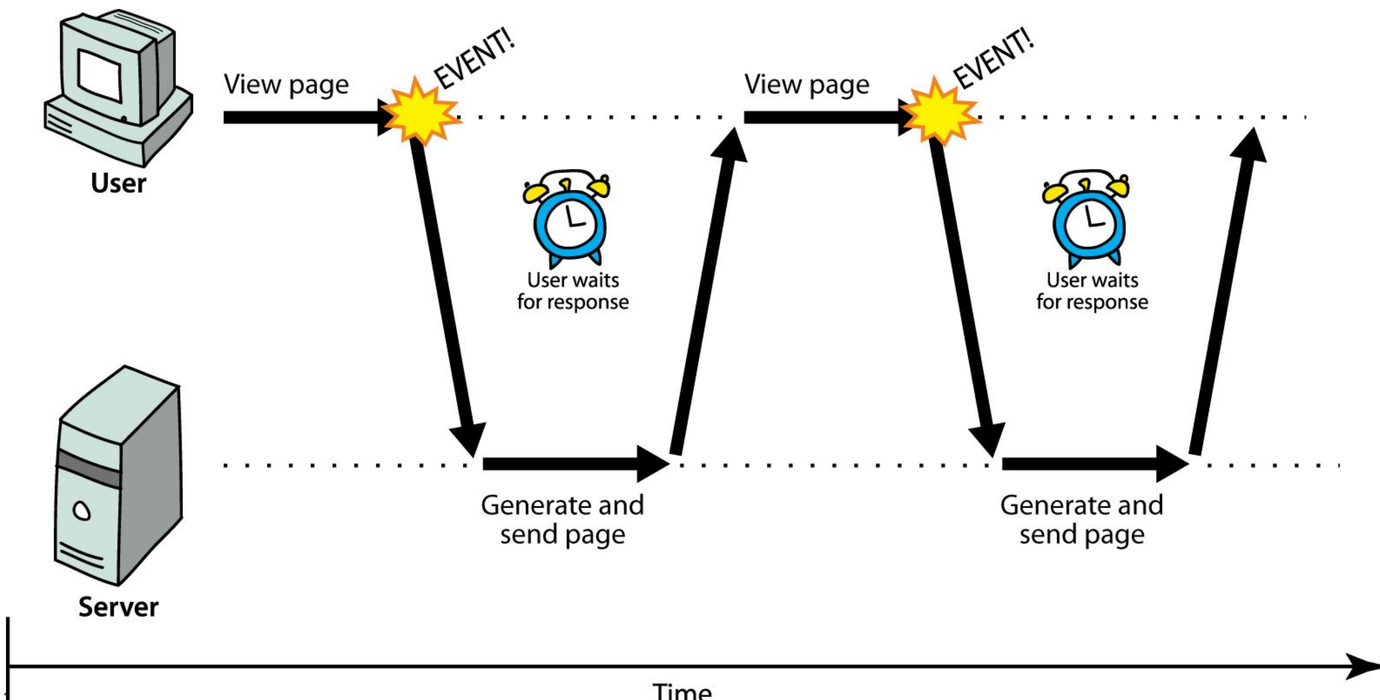
- A browser built-in `XMLHttpRequest` object (to send request to server)
- JavaScript and HTML DOM (to display or use response data)

With AJAX, we can:

- Read data from a web server, after the page has loaded
- Update a (part of) web page without reloading the page
- Send data to a web server in the background (i.e. update parts of a web page, without reloading the whole page)

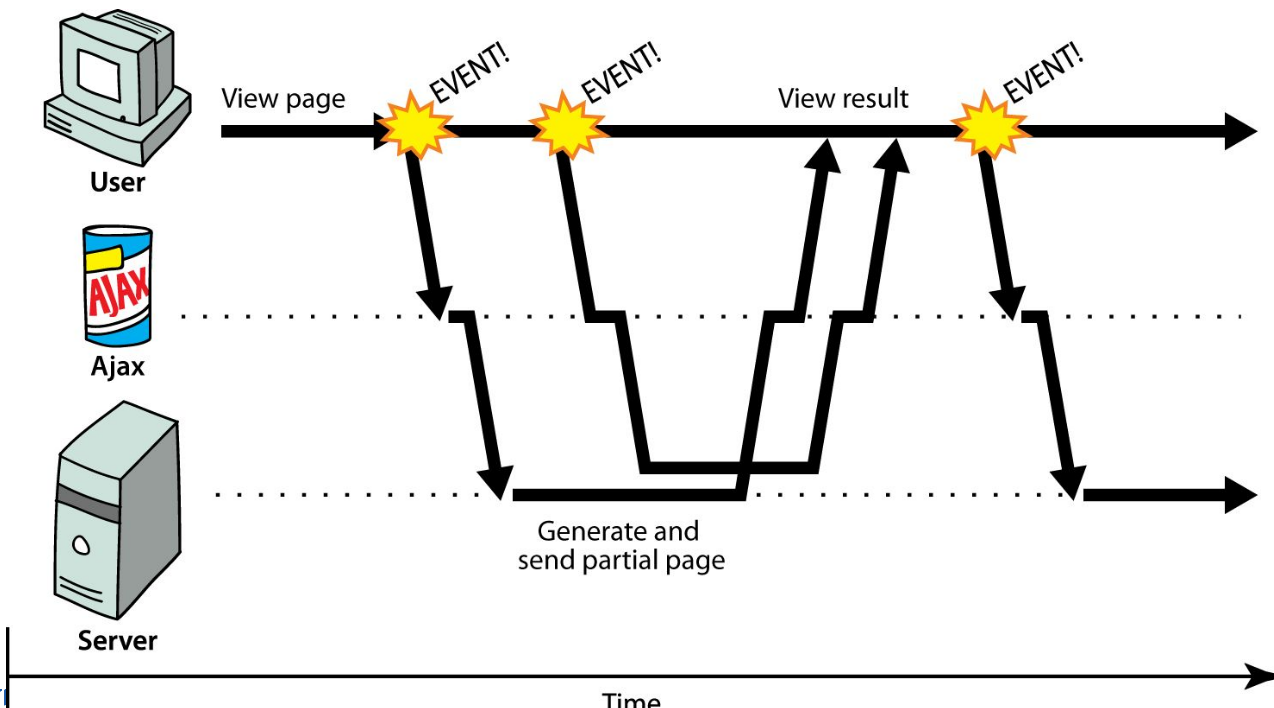
Synchronous Communication

User have to wait while browser communicates with the web server



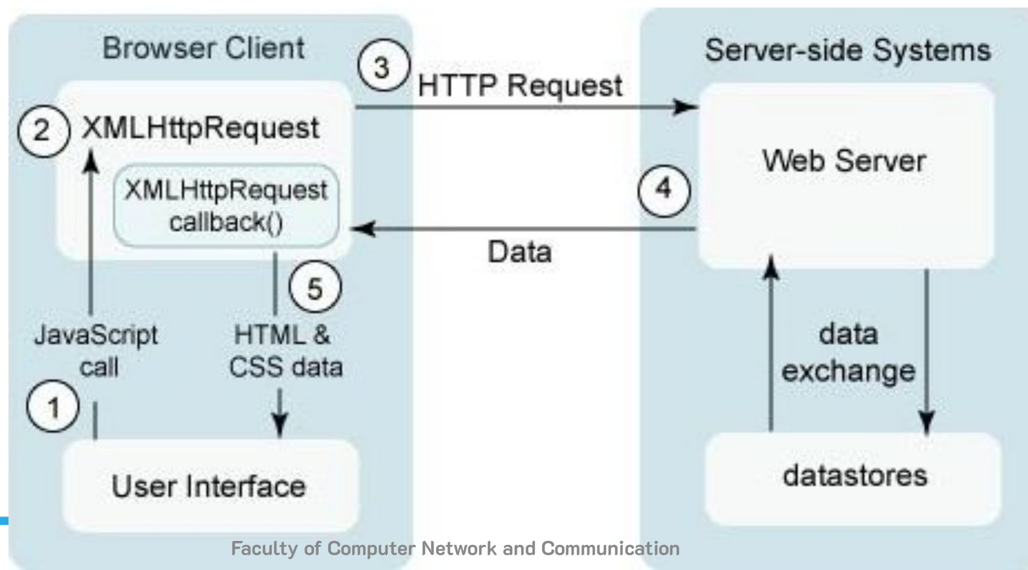
Asynchronous Communication

Users can interact with the web page while the browser is communicating with web server



How AJAX Works?

Browser		Server		Browser
An event occur... • Create an `XMLHttpRequest` object • Sent `HttpRequest`	Internet →	• Process `HttpRequest` • Create a response and send data back to the browser	Internet →	• Process the returned data using JS • Update page content



XMLHttpRequest Object

Used to exchange data with web server behind the scenes

Send request to server:

- `open(method, url, async)`: specify the type of the request
  - method: GET/POST
  - url: server/file location
  - async: true (asynchronous) or false (synchronous)
- `send()`: send the request to the server (used for GET)
- `send(string)`: send the request to the server (used for POST)
- `setRequestHeader(header, value)`: add HTTP header to the request

XMLHttpRequest Object

Request examples:

GET request:

```
var xhttp = new XMLHttpRequest();
xhttp.open("GET", "demo_get.php", true);
xhttp.send();
```

To avoid cached result:

```
xhttp.open("GET", "demo_get.php?t=" + Math.random(), true);
```

POST request:

```
xhttp.open("POST", "demo_post2.asp", true);
xhttp.setRequestHeader("Content-type", "application/x-www-form-urlencoded");
xhttp.send("fname=Henry&lname=Ford");
```

XMLHttpRequest Object

Request examples:

POST with JSON data:

```
var req = new XMLHttpRequest();
req.open("POST", "demo_post3", true);
req.setRequestHeader("Content-Type",
                     "application/json; charset=UTF-8");
var jsonBody = {
  "name": "An Tran",
  "url": "http://sites.google.com/site/tcanvn"
};

req.send(JSON.stringify(jsonBody));
```

XMLHttpRequest Object

Server response:

`readyState`: the status of the XMLHttpRequest object (0: request not initialized, 1: server connection established, 2: request received, 3: processing request, **4: response is ready**)

`onreadystatechange`: a function to be execute when the readyState changes

```
req.onreadystatechange = function() {
  if (this.readyState == 4 && this.status == 200) {
    aCallbackFunction(this);
  }
};
```

XMLHttpRequest Object

Server response:

- `status`, `statusText`: status of the `XMLHttpRequest` object (**200: "OK"**, 403: "Forbidden", 404: "Page not found")
- `responseText`: contains the response data as a string
- `responseXML`: contains the response data as XML data
- `getResponseHeader(header)`: returns specific header information from the server
- `getAllResponseHeaders()`: Returns all the header information from the server

loadDoc() Function

If there are more than one AJAX tasks, we should create:

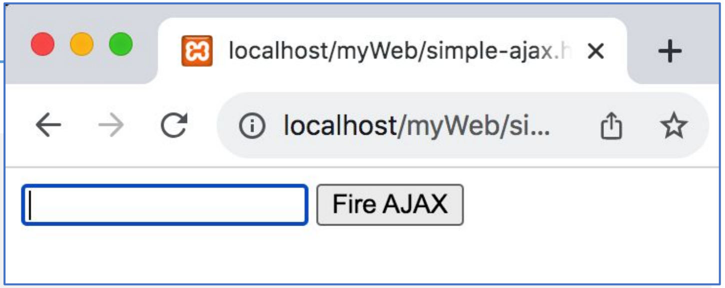
- A loadDoc() function for executing the XMLHttpRequest with two parameters: the requested URL and the callback function
- A callback function for each AJAX task with

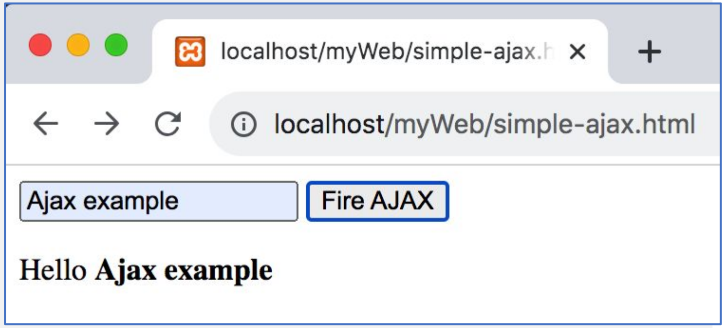
```
loadDoc("url1", callbackFunction1);  //AJAX task 1
loadDoc("url2", callbackFunction2);  //AJAX task 2...

function loadDoc(url, callbackFunction) {
  var xhttp = new XMLHttpRequest();
  xhttp.onreadystatechange = function() {
    if (this.readyState == 4 && this.status == 200) {
      callbackFunction(this);
    }
  };
  xhttp.open("GET", url, true);
  xhttp.send();
} //loadDoc()
```

```
function callbackFunctionX(xhttp) {
   // action goes here
}
```

AJAX Simple Example

```html
<html> <!-- simple-ajax.html -->
  <head>
    <script src="simple-ajax.js"></script>
  </head>

  <body>
    <form name="myform" action="">
      <input type="text" name="myname">
      <input type="button" name="mybutton"
             value="Fire AJAX"
             onclick="loadDoc('hello-name.php?name=' +
                      myform.myname.value, showHello);">
    </form>
    <p id="msg"></p>
  </body>
</html>
```





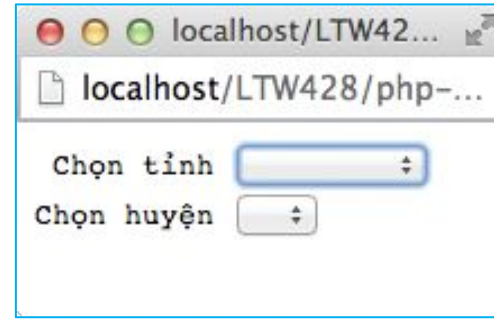
AJAX Simple Example

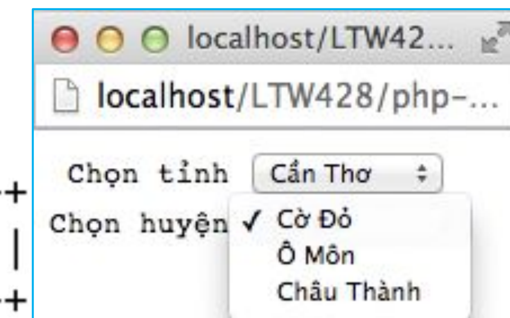
```
/* simple-ajax.js */
function loadDoc(url, callback) {
  var xmlhttp = new XMLHttpRequest();
  xmlhttp.onreadystatechange = function() {
    if (xmlhttp.readyState == 4 && xmlhttp.status == 200) {
      callback(this);
    }
  }
  xmlhttp.open("GET", url, true);
  xmlhttp.send();
}

function showHello(xmlhttp) {
  document.getElementById("msg").innerHTML = xmlhttp.responseText;
}
```

```
<?php   /* hello-name.php */

 if (isset($_GET['name'])) {
  echo "Hello <b>"
       . $_GET['name']
       . "</b>";
 }
?>
```

AJAX Example – Dropdown List

mahuyen	tenhuyen	matinh
1	Cờ Đỏ	1
2	Ô Môn	1
3	Châu Thành	1
4	Long Phú	2
5	Thạnh Trị	2
6	Ngã Năm	2
7	Trần Đề	2
8	Vĩnh Châu	3
9	Giá Rai	3

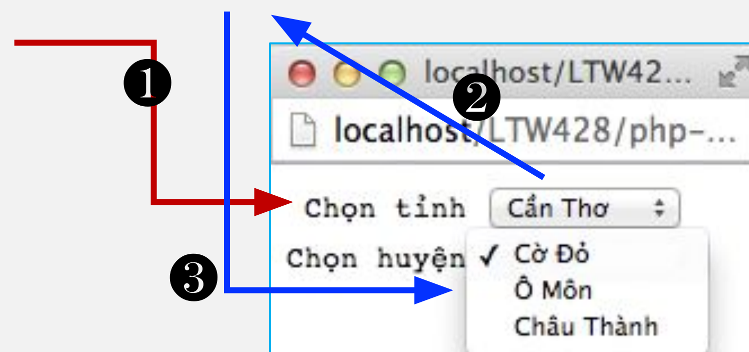




matinh	tentinh
1	Cần Thơ
2	Sóc Trăng
3	Bạc Liêu

AJAX Example – Dropdown List

```
<html>   <!-- dropdown-ajax.php -->
  <head>
    <meta charset="utf-8"/>
    <script src="dropdown-ajax-functions.js"></script>
  </head>
  <body>
    <?php require_once "dropdown-php-function.php"; ?>
    <form name="f_profile" action="" method="GET">
      <pre>
      Chọn tỉnh <select name="s_tinh" onchange="chonTinh(this.value)">
                  <?php gen_tinh_options(); ?>
                </select>
      Chọn huyện <select name="s_huyen"> </select>
      </pre>
    </form>
  </body>
</html>
```



AJAX Example – Dropdown List

```php
<?php   /* dropdown-php-function.php */
 function gen_tinh_options() {
   require_once "connect-select-db.php";
   $query = "SELECT * FROM tinh";
   $result = $conn->query($query)
     or die("SELECT failed: " . $conn->error);
   echo "<option value=''></option>";
   while ($row = $result->fetch_assoc()) {
     echo "<option value='" . $row['matinh'] . "'>"
         . $row['tentinh'] . "</option>";
   }
 }
?>
```

AJAX Example – Dropdown List

```
/* dropdown-ajax-function.js */
function chonTinh(matinh) {
  xmlhttp = new XMLHttpRequest();
  xmlhttp.onreadystatechange = function() {
    if (xmlhttp.readyState==4 && xmlhttp.status==200) {
      refreshHuyen(f_profile.s_huyen, xmlhttp.responseText);
    }
  }
  xmlhttp.open("GET", "get-huyen.php?matinh=" + matinh, true);
  xmlhttp.send();
}
```

AJAX Example – Dropdown List

```
/* dropdown-ajax-function.js */
function refreshHuyen(selectHuyen, data) {
  //data: maHuyen&&tenHuyen;;maHuyen&&tenHuyen;;...

  //remove all items
  var length = selectHuyen.options.length;
  for (i=0; i<length; i++)
    selectHuyen.remove(0);

  //add new data
  data = data.split(";;");
  for (i=0; i<data.length-1; i++) {
    var huyen = document.createElement("option");
    var itemData = data[i].split("&&");
    huyen.value = itemData[0]; huyen.text = itemData[1];
    selectHuyen.add(huyen);
  }
}
```

AJAX Example – Dropdown List

```php
<?php    /* get-huyen.php */

  if (isset($_GET['matinh'])) {
    require_once "connect-select-db.php";
    $matinh = $_GET['matinh'];
    $result = $conn->query("SELECT * FROM huyen WHERE matinh=$matinh")
      or die("Retrieve data failed: " . $conn->error);

    while ($row = $result->fetch_assoc()) {
      echo "$row[mahuyen]&&$row[tenhuyen];;";
    }
  }
?>
```

Exercise:
Using JSON format

get-huyen.php:
- set content-type to JSON (header)
- use json_encode() to encode data

refresh-huyen.php:
- use JSON.parse() to decode data

Content

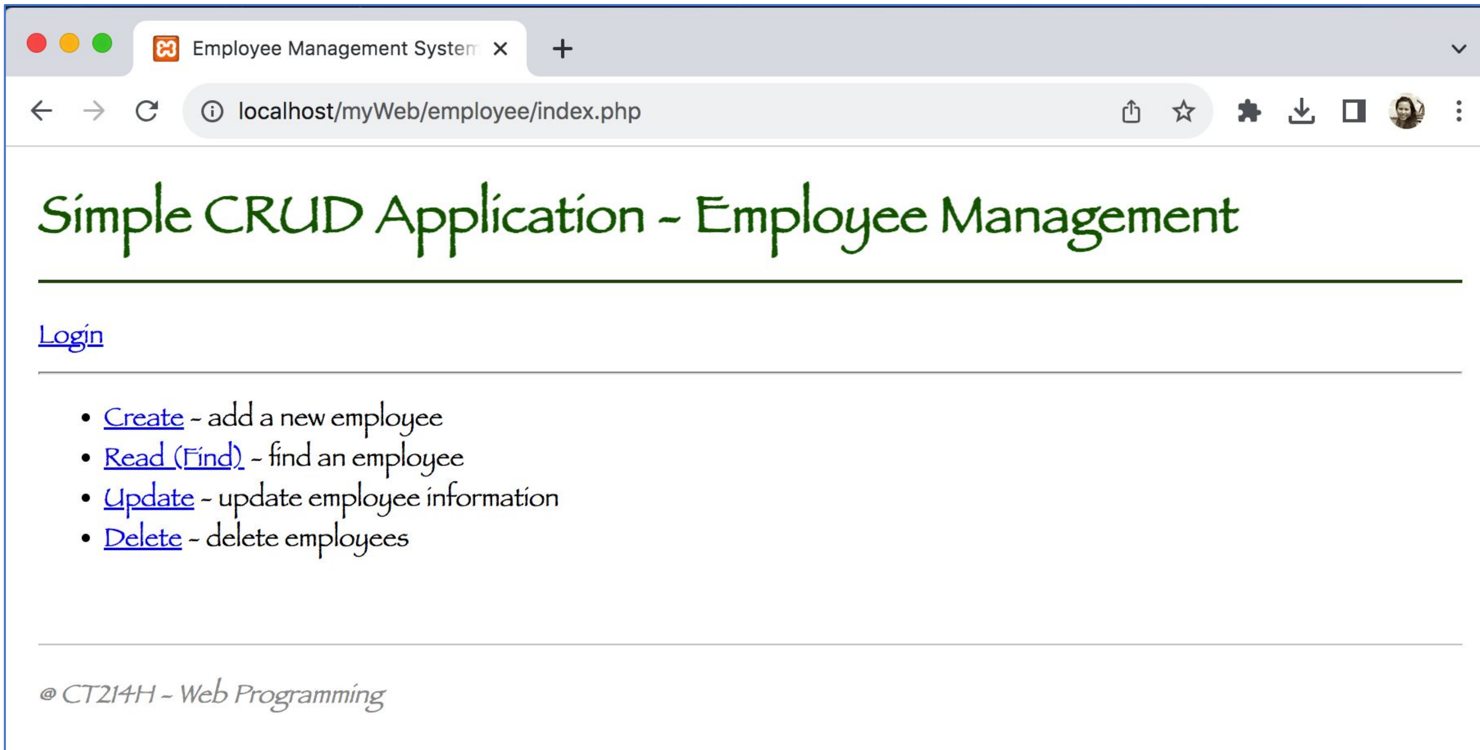
1. Introduction
2. PHP language basics
3. PHP functions
4. OOP in PHP
5. PHP and forms
6. PHP and MySQL
7. Cookies and sessions
8. Advanced PHP techniques (file upload, pagination, AJAX, etc.)
9. Appendix

Appendix

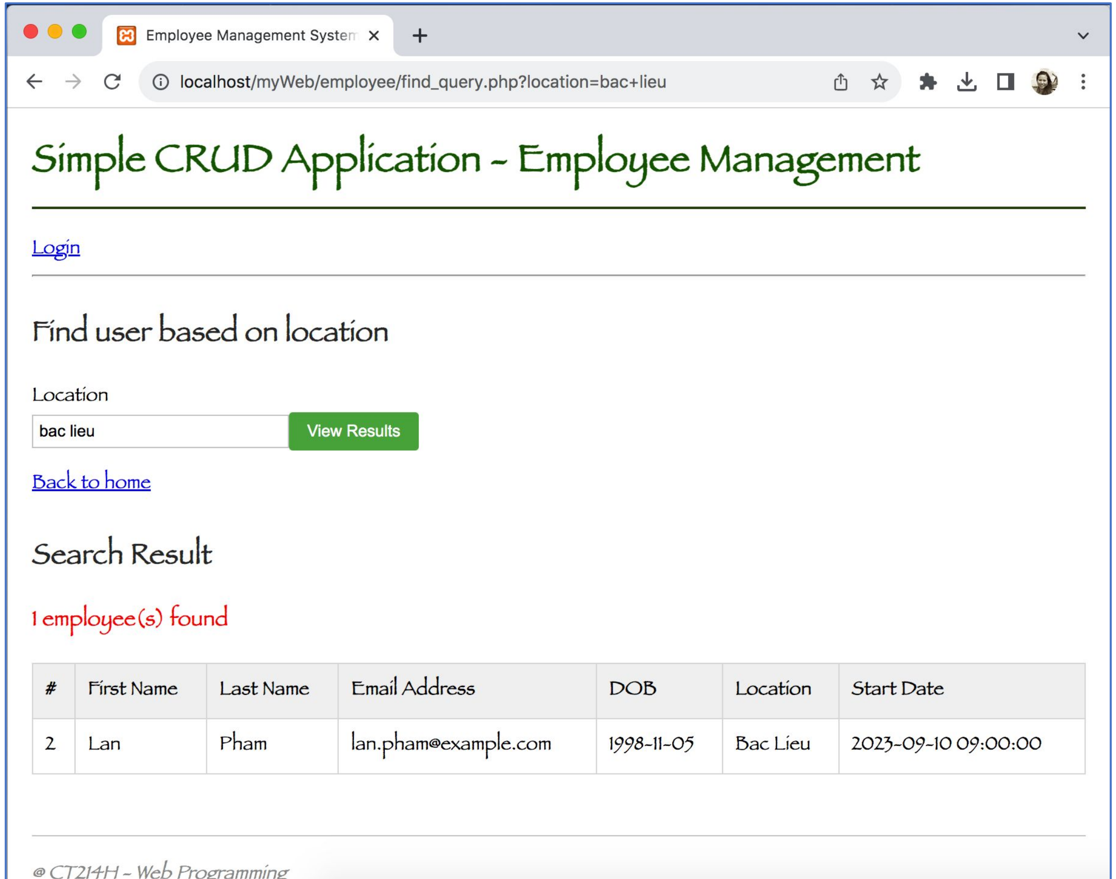
A simple CRUD "Employee Manager Application"

Application GUI

Appendix – Simple CRUD Application



Application GUI



Employee Management System

localhost/myWeb/employee/find_query.php?location=bac+lieu

Simple CRUD Application - Employee Management

Login

Find user based on location

Location

bac lieu

View Results

Back to home

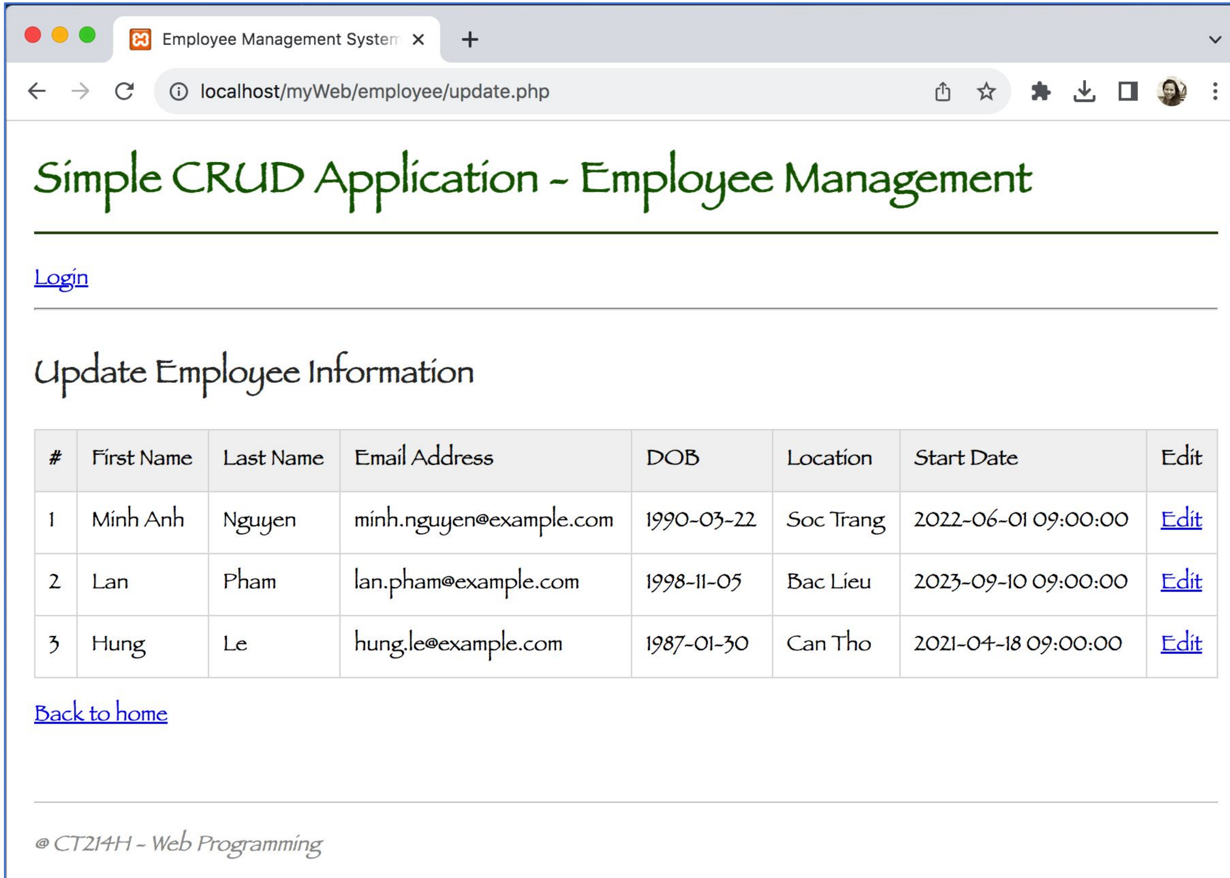
Search Result

1 employee(s) found

#	First Name	Last Name	Email Address	DOB	Location	Start Date
2	Lan	Pham	lan.pham@example.com	1998-11-05	Bac Lieu	2023-09-10 09:00:00

@ CT214H - Web Programming

Application GUI



Simple CRUD Application - Employee Management

Login

Update Employee Information

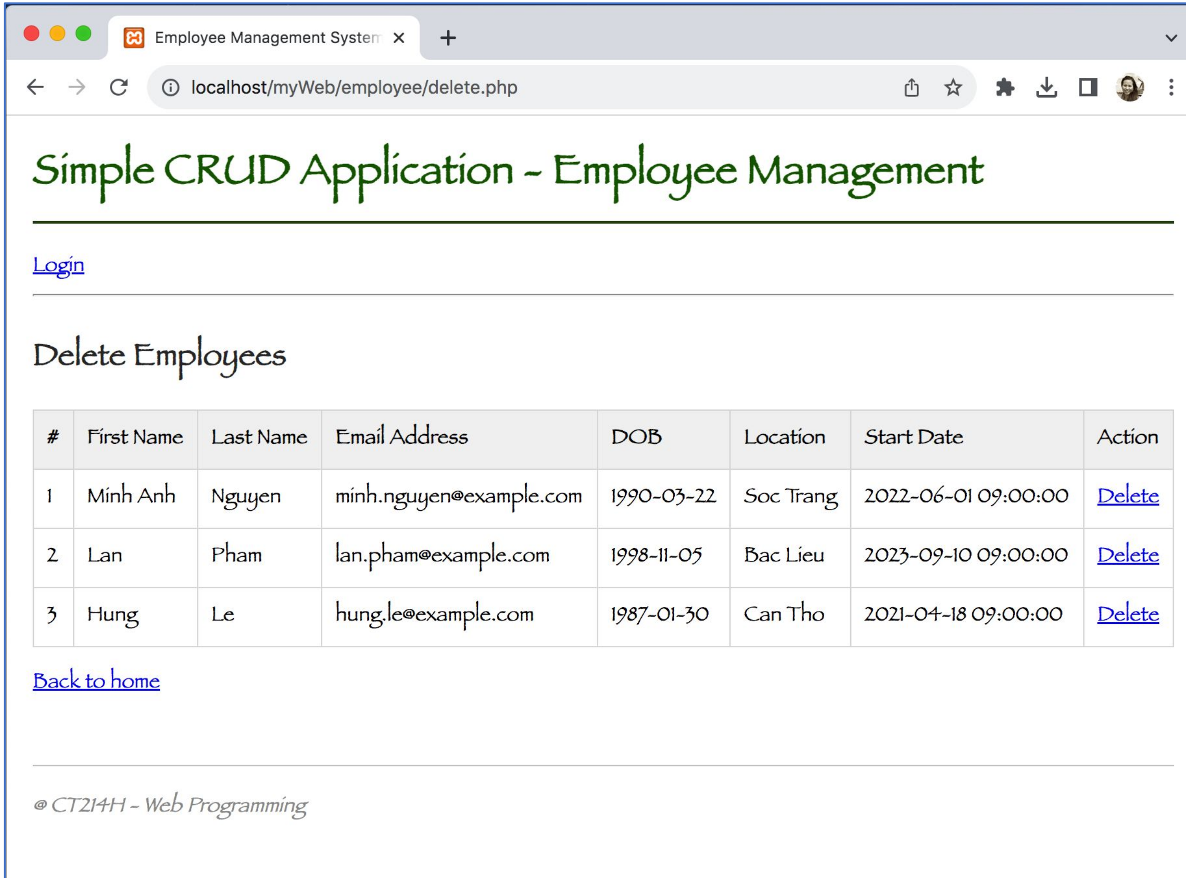
#	First Name	Last Name	Email Address	DOB	Location	Start Date	Edit
1	Minh Anh	Nguyen	minh.nguyen@example.com	1990-03-22	Soc Trang	2022-06-01 09:00:00	Edit
2	Lan	Pham	lan.pham@example.com	1998-11-05	Bac Lieu	2023-09-10 09:00:00	Edit
3	Hung	Le	hung.le@example.com	1987-01-30	Can Tho	2021-04-18 09:00:00	Edit

Back to home

@ CT214H - Web Programming

• Appendix – Simple CRUD Application

Application GUI



Application and DB Structure

```
employee/
│
├── css/
│   └── style.css
│
├── data/
│   ├── connect-select-db.php
│   └── db-schema.sql
│
├── templates/
│   ├── header.php
│   └── footer.php
│
├── create.php
├── delete.php
├── find.php
├── find_query.php
├── index.php
├── update.php
└── update-single.php
```

```sql
CREATE TABLE employee (
    id INT(11) UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    firstname VARCHAR(30) NOT NULL,
    lastname VARCHAR(30) NOT NULL,
    email VARCHAR(50) NOT NULL,
    dob DATE,
    location VARCHAR(50),
    startdate TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO employee (firstname, lastname, email, dob, location, startdate)
VALUES
('Minh', 'Nguyen', 'minh.nguyen@example.com', '1990-03-22', 'Soc Trang', '2022-06-01 09:00:00'),
('Lan', 'Pham', 'lan.pham@example.com', '1998-11-05', 'Bac Lieu', '2023-09-10 09:00:00'),
('Hung', 'Le', 'hung.le@example.com', '1987-01-30', 'Can Tho', '2021-04-18 09:00:00');
```

Connect to MySQL

connect-select-db.php

```
<?php
	$host = 'localhost';
	$user = 'root';
	$password = '';
	$database = 'myDB';

	$conn = mysqli_connect($host, $user, $password, $database);

	if (!$conn) {
	    die("Connection failed: " . mysqli_connect_error());
	}

	mysqli_set_charset($conn, "utf8");
?>
```

Header and Footer

header.php

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Employee Management System</title>
    <link rel="stylesheet" href="css/style.css">
</head>
<body>
    <h1>Simple CRUD Application - Employee Management</h1>
    <a href="index.php">Login</a>
```

footer.php

```
<div class="footer">
        @ CT214H - Web Programming
    </div>
</body>
</html>
```

[style.css]

Index Page (index.php)

```
<?php include 'templates/header.php'; ?>

<ul>
    <li><a href="create.php">Create</a> - add a new employee</li>
    <li><a href="find.php">Read (Find)</a> - find an employee</li>
    <li><a href="update.php">Update</a> - update employee information</li>
    <li><a href="delete.php">Delete</a> - delete employees</li>
</ul>

<?php include 'templates/footer.php'; ?>
```

Adding New Employees (create.php)

```
<?php
include 'templates/header.php';
include 'data/connect-select-db.php';

if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $firstname = mysqli_real_escape_string($conn, $_POST['firstname']);
    $lastname = mysqli_real_escape_string($conn, $_POST['lastname']);
    $email = mysqli_real_escape_string($conn, $_POST['email']);
    $dob = mysqli_real_escape_string($conn, $_POST['dob']);
    $location = mysqli_real_escape_string($conn, $_POST['location']);

    $sql = "INSERT INTO employee (firstname, lastname, email, dob, location)
            VALUES ('$firstname', '$lastname', '$email', '$dob', '$location')";

    if (mysqli_query($conn, $sql)) {
        echo "<div class='message success'>Employee added successfully!</div>";
    } else {
        echo "<div class='message error'>Error: " . mysqli_error($conn) . "</div>";
    }
} ?>
```

Adding New Employees (create.php)

```
<h2>Create New Employee</h2>
<form method="POST">
    <table>
        <tr><td>First Name:</td>
            <td><input type="text" name="firstname" required></td> </tr>
        <tr><td>Last Name:</td>
            <td><input type="text" name="lastname" required></td></tr>
        <tr><td>Email:</td>
            <td><input type="email" name="email" required></td></tr>
        <tr><td>Date of Birth:</td>
            <td><input type="date" name="dob"></td></tr>
        <tr><td>Location:</td>
            <td><input type="text" name="location"></td></tr>
        <tr><td></td>
            <td><input type="submit" value="Create Employee"></td></tr>
    </table>
</form>
<p><a href="index.php">Back to home</a></p>
<?php   mysqli_close($conn);
include 'templates/footer.php';  ?>
```

Find an Employee (find.php)

```
<?php
include 'templates/header.php';
include 'data/connect-select-db.php';
?>

<h2>Find user based on location</h2>
<form method="GET" action="find_query.php">
    <label>Location</label><br>
    <input type="text" name="location" size="20">
    <input type="submit" value="View Results">
</form>

<p><a href="index.php">Back to home</a></p>

<?php
mysqli_close($conn);
include 'templates/footer.php';
?>
```

Find an Employee (find-query.php)

```
<?php
include 'templates/header.php';
include 'data/connect-select-db.php';
echo "<h2>Find user based on location</h2>";
echo "<form method='GET' action='find_query.php'>";
echo "<label>Location</label><br>";
echo "<input type='text' name='location' value='"
     . htmlspecialchars($_GET['location'] ?? '') . "' size='20'>";
echo "<input type='submit' value='View Results'>";
echo "</form>";
echo "<p><a href='index.php'>Back to home</a></p>";

if (isset($_GET['location']) && !empty($_GET['location'])) {
    $location = mysqli_real_escape_string($conn, $_GET['location']);
    $sql = "SELECT * FROM employee WHERE location LIKE '%$location%'";
    $result = mysqli_query($conn, $sql);
    $count = mysqli_num_rows($result);
    echo "<h2>Search Result</h2>";
    echo "<p class='result-count'>$count employee(s) found</p>";
```

Find an Employee (find-query.php)

```
    if ($count > 0) {
        echo "<table>";
        echo "<tr><th>#</th> <th>First Name</th><th>Last Name</th>
                <th>Email Address</th> <th>DOB</th><th>Location</th>
                <th>Start Date</th> </tr>";
        while ($row = mysqli_fetch_assoc($result)) {
            echo "<tr>";
            echo "<td>" . $row['id'] . "</td>";
            echo "<td>" . htmlspecialchars($row['firstname']) . "</td>";
            echo "<td>" . htmlspecialchars($row['lastname']) . "</td>";
            echo "<td>" . htmlspecialchars($row['email']) . "</td>";
            echo "<td>" . $row['dob'] . "</td>";
            echo "<td>" . htmlspecialchars($row['location']) . "</td>";
            echo "<td>" . $row['startdate'] . "</td>";
            echo "</tr>";
        }
        echo "</table>";
    }
}
mysqli_close($conn);
include 'templates/footer.php'; ?>
```

Update Form Generation (update.php)

```php
<?php include 'templates/header.php'; include 'data/connect-select-db.php';
echo "<h2>Update Employee Information</h2>";
$sql = "SELECT * FROM employee ORDER BY id";
$result = mysqli_query($conn, $sql);
if (mysqli_num_rows($result) > 0) {
    echo "<table>";
    echo "<tr><th>#</th><th>First Name</th><th>Last Name</th>
            <th>Email Address</th><th>DOB</th><th>Location</th>
            <th>Start Date</th><th>Edit</th></tr>";
    while ($row = mysqli_fetch_assoc($result)) {
        echo "<tr>";
        echo "<td>" . $row['id'] . "</td>";
        echo "<td>" . htmlspecialchars($row['firstname']) . "</td>";
        echo "<td>" . htmlspecialchars($row['lastname']) . "</td>";
        echo "<td>" . htmlspecialchars($row['email']) . "</td>";
        echo "<td>" . $row['dob'] . "</td>";
        echo "<td>" . htmlspecialchars($row['location']) . "</td>";
        echo "<td>" . $row['startdate'] . "</td>";
        echo "<td><a href='update-single.php?id=" . $row['id'] . "'>Edit</a></td>";
        echo "</tr>";
    }
    echo "</table>";
} else { echo "<p>No employees found.</p>"; }
echo "<p><a href='index.php'>Back to home</a></p>";
mysqli_close($conn); include 'templates/footer.php'; ?>
```

Update Single Employee (update-single.php)

```
<?php include 'templates/header.php'; include 'data/connect-select-db.php';
$id = mysqli_real_escape_string($conn, $_GET['id']);
if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $firstname = mysqli_real_escape_string($conn, $_POST['firstname']);
    $lastname = mysqli_real_escape_string($conn, $_POST['lastname']);
    $email = mysqli_real_escape_string($conn, $_POST['email']);
    $dob = mysqli_real_escape_string($conn, $_POST['dob']);
    $location = mysqli_real_escape_string($conn, $_POST['location']);
    $sql = "UPDATE employee SET firstname='$firstname',
        lastname='$lastname', email='$email', dob='$dob',
        location='$location' WHERE id='$id'";
    if (mysqli_query($conn, $sql)) {
        echo "<div class='message success'>Employee updated successfully!</div>";
        header("Location: update.php");
        exit();
    } else {
        echo "<div class='message error'>Error: " . mysqli_error($conn) . "</div>";
    }
}
$sql = "SELECT * FROM employee WHERE id='$id'";
$result = mysqli_query($conn, $sql);
$employee = mysqli_fetch_assoc($result);
if (!$employee) { echo "<p>Employee not found.</p>"; exit(); } ?>
```

Update Single Employee (update-single.php)

```
<h2>Update Employee #<?php echo $id; ?></h2>
<form method="POST">
    <table>
        <tr><td>First Name:</td><td><input type="text" name="firstname"
            value="<?php echo htmlspecialchars($employee['firstname']); ?>" required></td></tr>
        <tr><td>Last Name:</td><td><input type="text" name="lastname"
            value="<?php echo htmlspecialchars($employee['lastname']); ?>" required></td></tr>
        <tr><td>Email:</td><td><input type="email" name="email"
            value="<?php echo htmlspecialchars($employee['email']); ?>" required></td></tr>
        <tr><td>Date of Birth:</td><td><input type="date" name="dob"
            value="<?php echo $employee['dob']; ?>"></td></tr>
        <tr><td>Location:</td><td><input type="text" name="location"
        value="<?php echo htmlspecialchars($employee['location']); ?>"></td></tr>
        <tr><td></td><td><input type="submit" value="Update Employee">
            <a href="update.php">Cancel</a></td></tr>
    </table>
</form>
<p><a href="index.php">Back to home</a></p>
<?php
mysqli_close($conn);
include 'templates/footer.php';
?>
```

Delete Employee (delete.php)

```
<?php
include 'templates/header.php';
include 'data/connect-select-db.php';
if (isset($_GET['id'])) {
    $id = mysqli_real_escape_string($conn, $_GET['id']);
    $sql = "DELETE FROM employee WHERE id='$id'";

    if (mysqli_query($conn, $sql)) {
        echo "<div class='message success'>Employee deleted successfully!</div>";
    } else {
        echo "<div class='message error'>Error: " . mysqli_error($conn) . "</div>";
    }
}
echo "<h2>Delete Employees</h2>";
$sql = "SELECT * FROM employee ORDER BY id";
$result = mysqli_query($conn, $sql);
```

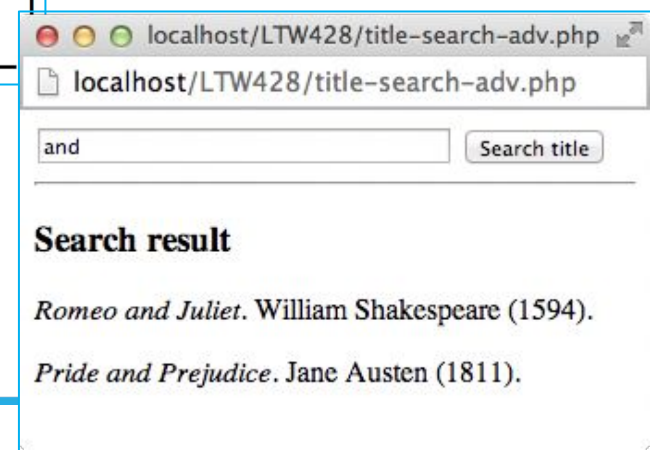
Delete Employee (delete.php)

```
if (mysqli_num_rows($result) > 0) {
    echo "<table>";
    echo "<tr><th>#</th><th>First Name</th><th>Last Name</th>
        <th>Email Address</th><th>DOB</th><th>Location</th>
        <th>Start Date</th><th>Action</th></tr>";
    while ($row = mysqli_fetch_assoc($result)) {
        echo "<tr>";
        echo "<td>" . $row['id'] . "</td>";
        echo "<td>" . htmlspecialchars($row['firstname']) . "</td>";
        echo "<td>" . htmlspecialchars($row['lastname']) . "</td>";
        echo "<td>" . htmlspecialchars($row['email']) . "</td>";
        echo "<td>" . $row['dob'] . "</td>";
        echo "<td>" . htmlspecialchars($row['location']) . "</td>";
        echo "<td>" . $row['startdate'] . "</td>";
        echo "<td><a href='delete.php?id=" . $row['id'] . "'
            onclick='return confirm(\"Are you sure?\")'>Delete</a></td>";
        echo "</tr>";
    }
    echo "</table>";
} else { echo "<p>No employees found.</p>"; }
echo "<p><a href='index.php'>Back to home</a></p>";
mysqli_close($conn);
include 'templates/footer.php'; ?>
```

Exercises

Exercise 1 – Book Search

```
MariaDB [ct219h]> describe classics;
+--------+----------------+------+-----+---------+
| Field  | Type           | Null | Key | Default |
+--------+----------------+------+-----+---------+
| isbn   | char(13)       | NO   | PRI | NULL    |
| author | varchar(128)   | YES  | MUL | NULL    |
| title  | varchar(128)   | YES  |     | NULL    |
| type   | varchar(16)    | YES  |     | NULL    |
| year   | char(4)        | YES  |     | NULL    |
| price  | float          | YES  |     | 0       |
+--------+----------------+------+-----+--------
```



Exercise 1 – Book Search

Webpage and form:

```
<html>  <!-- title-search-adv.php -->
  <body>
    <form action="title-search-adv.php" method="POST">
      <input type="text" size="40" name="search_kw"
        value="<?php if (!empty($_POST['search_kw']))
          echo $_POST['search_kw'];?>"/>
      <input type="submit" value="Search title">
      <hr>
    </form>
    <h3>Search result</h3>
    <?php
      if (isset($_POST['search_kw'])) {
         include 'title-search-func.php';
         search($_POST['search_kw']);  //defined in title-search-func.php
      }
    ?>
  </body>
</html>
```

Exercise 1 – Book Search

Search function:

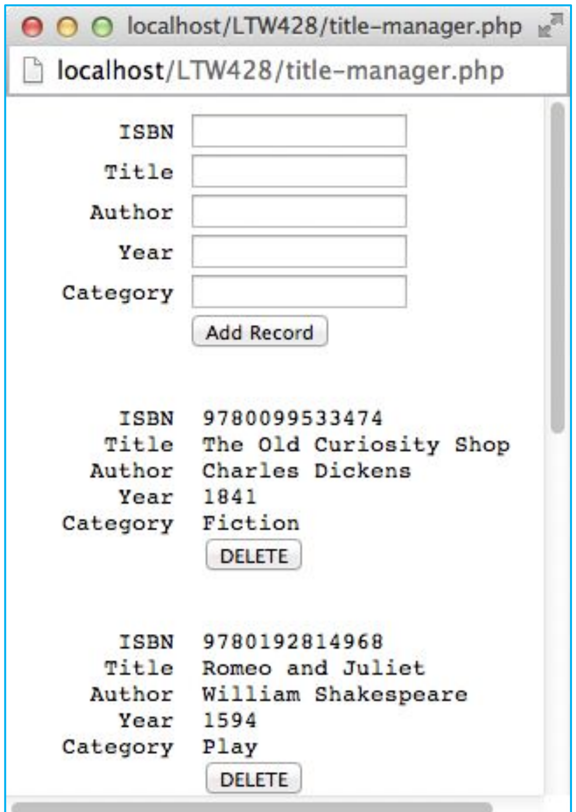
```
<?php    /* title-search-func.php */
  function search($keyword) {
    require "connect-select-db.php";
    $keyword = trim($keyword);
    $new_kw  = str_replace(" ", "%' OR title LIKE '%", $keyword);
    $query   = "SELECT * FROM classics WHERE title LIKE '%$new_kw%'";

    $result = $conn->query($query);
    if ($result->num_rows > 0) {
      while ($row = $result->fetch_assoc()) {
         echo "<p><i>Title: " . $row["title"] . "</i> - Author: " .
       $row["author"]. " (" . $row["year"]. ")</p><br>";}
    }
    else
      echo "No title found";
  }
?>
```

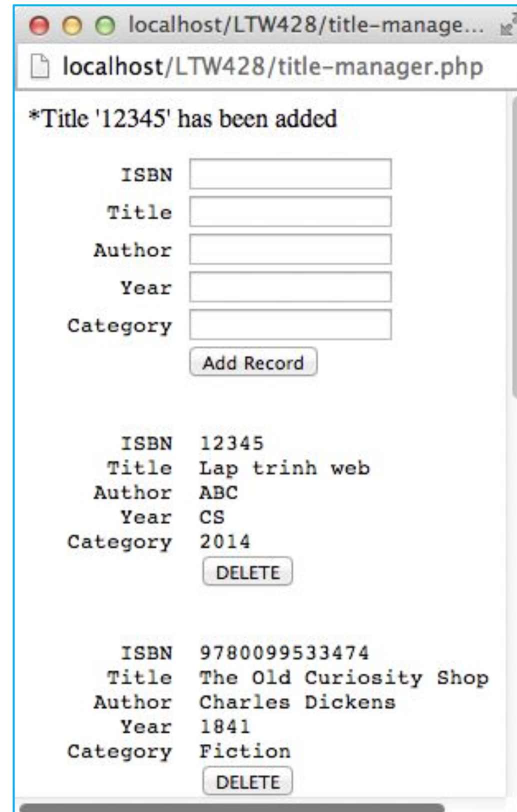
Exercise 1 – Book Search

Search function: => try search with paging

Exercise 2 – Title Management

Application interface:





Exercise 2– Title Management

Main page:

```php
<?php   /* title-management.php */

  require "connect-select-db.php"; //connect to mySQL and provide $conn
  require "title-delete-process.php";   //delete a title
  require "title-add-process.php";      //add new title
  require "title-add-form.php";         //create add-title form
  require "title-list-delete-form.php"; //function del_form_gen()

  //retrieve all records (to create [delete title] forms)
  $query  = "SELECT * FROM classics";
  $result = $conn->query($query)
               or die("DB Access error: " . mysql_error());

  //generate [delete title] form
  while ($row = $result->fetch_assoc()) {
    del_form_gen($row);
  }
  $conn->close(); //close connection
?>
```

Exercise 2 – Title Management

Add title form:

```
<?php    /* title-add-form.php */
  echo <<<_ADD_TITLE_FORM
  <form action="title-manager.php" method="POST">
   <pre>
      ISBN <input type="text" name="isbn"/>
     Title <input type="text" name="title"/>
    Author <input type="text" name="author"/>
      Year <input type="text" name="year"/>
  Category <input type="text" name="type"/>
           <input type="submit" value="Add Record">
    </pre>
    <input type="hidden" name="add" value="yes">
  </form>
_ADD_TITLE_FORM;
?>
```

Exercise 2 – Title Management

List and delete title form:

```
<?php    /* title-list-delete-form.php */
  function del_form_gen($row) {
     echo <<<_DEL_TITLE_FORM
     <form action="title-manager.php" method="POST" >
     <pre>
      ISBN $row[isbn]
     Title $row[title]
    Author $row[author]
      Year $row[year]
  Category $row[type]
           <input type="submit" value="DELETE">
     </pre>
     <input type="hidden" name="delete" value="yes">
     <input type="hidden" name="isbn" value="$row[isbn]">
  </form>

  _DEL_TITLE_FORM;
  } //add_form_gen()
?>
```

Confirm before deleting:

```
onsubmit="return confirm('Do you really want to delete the book?');"
```

Exercise 2 – Title Management

Process the delete title function:

```php
<?php    /* title-delete-process.php */
  if (isset($_POST['delete']) && isset($_POST['isbn'])) {
    $isbn = $_POST['isbn'];
    $query = "DELETE FROM classics WHERE isbn='$isbn'";

    if (!$conn->query($query)) {
      echo "<h3> DELETE failed: " . $isbn . ". Error: "
           . $conn->error . "</h3>";
    }
    else {
      echo "*Title '$isbn' has been deleted<br>";
    }
  }
  ?>
```

To show an alert box about the deletion:

```php
echo <<<_DEL_OK_PROMPT
  <script>
    alert("*Title '$isbn' has been deleted");
  </script>
  _DEL_OK_PROMPT;
```

Exercise 2 – Title Management

Process the add title function:

```php
<?php    /* title-add-process */
  if (isset($_POST["add"])) {
     $isbn = $_POST["isbn"];
     $author = $_POST["author"];
     $title = $_POST["title"];
     $year = $_POST["year"];
     $type = $_POST["type"];
     $query = "INSERT INTO classics VALUES "
         . "('$isbn', '$author', '$title', '$type', '$year', 0)";

     if (!$conn->query($query))
        echo "<h3>INSERT failed. " . $conn->error . "</h3>";
     else
        echo "*Title '$isbn' has been added<br>";
  }
?>
```

Exercise 2– Edit Book Details

Add the "Edit Book Detail" for the Book Manager Application as follow:

- Add an Edit button next to the Delete button
- When the user click the Edit button, open a new page that allows user to edit the details of the selected book (update page)
- In the update page, user can
  - Edit and Save the change
  - Go back to the home page

Exercise 3– Examples

Work on the examples of:

- File upload
- Ajax dropdown list

The End!